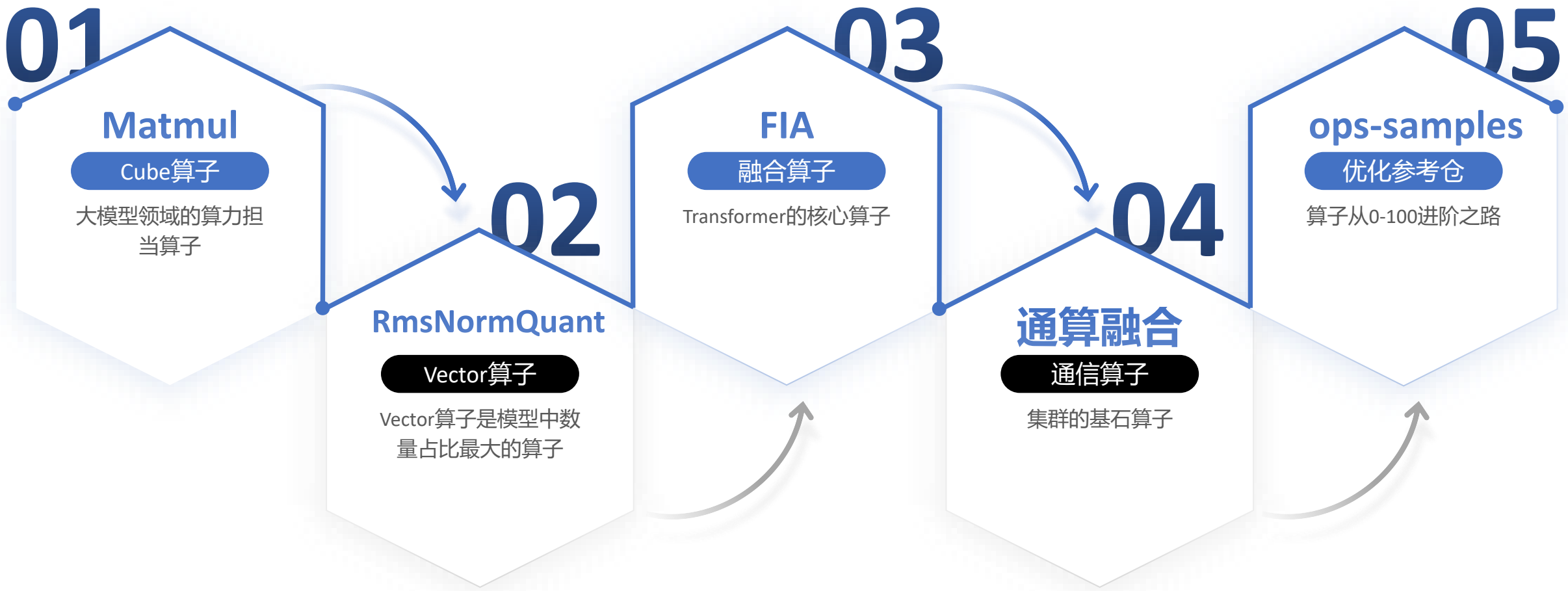
探索Ascend 950的性能天花板

作者：沈敏/俞郑中/唐超力/范照云

时间：2026/3/20


01
Matmul
Cube算子
大模型领域的算力担当算子
02
RmsNormQuant
Vector算子
Vector算子是模型中数量占比最大的算子
03
FIA
融合算子
Transformer的核心算子
04
通算融合
通信算子
集群的基石算子
05
ops-samples
优化参考仓
算子从0-100进阶之路

目录

Part 1 Matmul性能极致优化

Part 2 RmsnormQuant性能极致优化

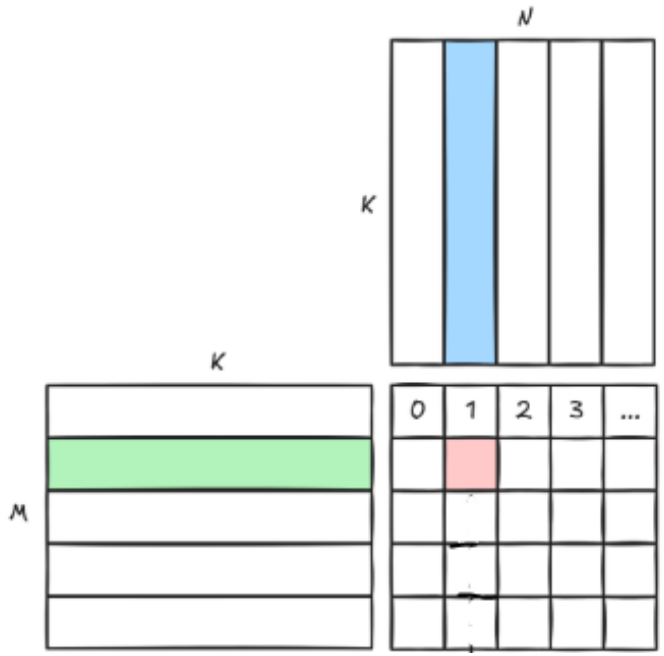
Part 3 FA类算子性能极致优化

Part 4 通信算子性能极致优化

Part 5 ops-samples仓介绍

Matmul算子功能介绍

计算图



功能介绍

- **简介：** Matmul（矩阵乘法）算子是深度学习、线性代数和科学计算中的核心计算单元，用于实现两个矩阵的乘法。它常用于神经网络的线性层，通过对激活矩阵A和权重矩阵B进行计算输出相应结果C。

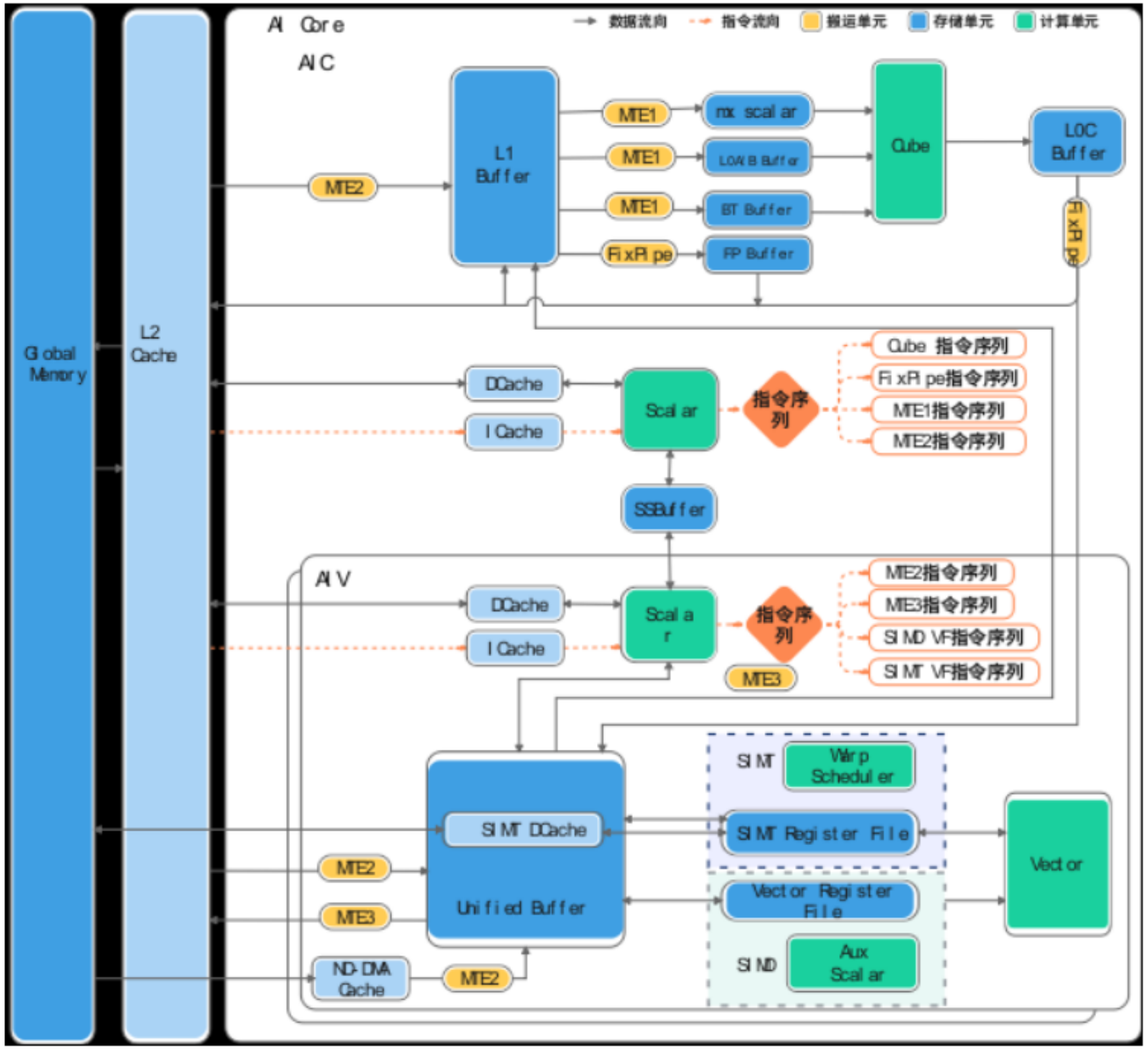
- **主要参数：**
  - m: 矩阵乘中左矩阵的行。
  - k: 矩阵乘中左矩阵的列/右矩阵的行。
  - n: 矩阵乘中右矩阵的列。

计算公式

$$C[m, n] = A[m, k]@B[k, n] + C[n]$$

CANN

Ascend 950芯片与Cube算子相关变化点



Ascend 950芯片与Cube算子相关变化点

算力：

①Cube单die核数相比上一代增加，算力持续提升

②Cube vs Vector 算力比持续提升

Local buffer：

①Cube L0C buffer 相比上一代翻倍

新增数据类型：

①Cube 新增低比特数据格式HiFP8， MXFP8/4, FP8

访存：

①Ascend 950DT支持片上内存访存带宽与内存容量相比上一代增加

②Sector Cache持续优化，提升非对齐/小包访问性能

数据流&同步：

①支持Cube和Vector Local buffer的直接CV数据传递通道

②支持BufferID消除原有同步set和wait之间的强制配对需求，简化同步实现

Sliding Window Adaptive Tiling技术

传统IterateK：行优先/列优先

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5
2	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
3	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11
4	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

方案实现简单，但L2命中率低。

上一代芯片方案：错位分核

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	0	7	14	21	28	3	10	26	1	8	15	22	29	13	20	27	2	9	16
1	17	1	8	15	22	29	4	11	27	2	9	16	23	30	14	21	28	3	10
2	11	18	2	9	16	23	30	5	12	28	3	10	17	24	31	15	22	29	4
3	5	12	19	3	10	17	24	31	6	13	29	4	11	18	25	0	16	23	30
4	31	6	13	20	4	11	18	25	0	7	14	30	5	12	19	26	1	17	24
5	25	0	7	14	21	5	12	19	26	1	8	15	31	6	13	20	27	2	18
6	19	26	1	8	15	22	6	13	20	27	2	9	16	0	7	14	21	28	3
7	4	20	27	2	9	16	23	7	14	21	28	3	10	17	1	8	15	22	29
8	30	5	21	28	3	10	17	24	8	15	22	29	4	11	18	2	9	16	23
9	24	31	6	22	29	4	11	18	25	9	16	23	30	5	12	19	3	10	17
10	18	25	0	7	23	30	5	12	19	26	10	17	24	31	6	13	20	4	11
11	12	19	26	1	8	24	31	6	13	20	27	11	18	25	0	7	14	21	5
12	6	13	20	27	2	9	25	0	7	14	21	28	12	19	26	1	8	15	22

改善数据集中访问，L2命中率不高，首次MMAD计算时综合带宽供应不足；该方案依赖L2 Cache大小。

新方案：SWAT

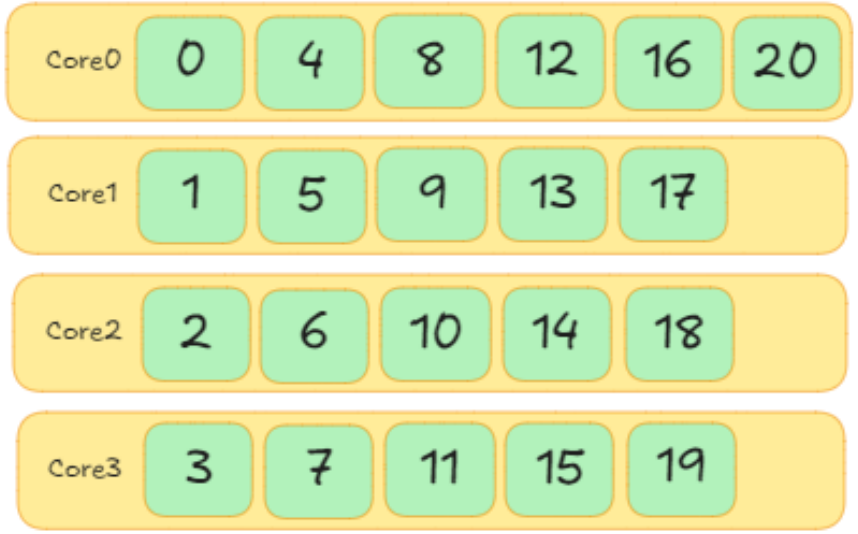
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	0	4	8	12	16	20	24	28	0	4	8	12	16	20	24	28	0	4	8
1	1	5	9	13	17	21	25	29	1	5	9	13	17	21	25	29	1	5	9
2	2	6	10	14	18	22	26	30	2	6	10	14	18	22	26	30	2	6	10
3	3	7	11	15	19	23	27	31	3	7	11	15	19	23	27	31	3	7	11
4	20	16	12	8	4	0	28	24	20	16	12	8	4	0	28	24	20	16	12
5	21	17	13	9	5	1	29	25	21	17	13	9	5	1	29	25	21	17	13
6	22	18	14	10	6	2	30	26	22	18	14	10	6	2	30	26	22	18	14
7	23	19	15	11	7	3	31	27	23	19	15	11	7	3	31	27	23	19	15
8	24	29	2	7	12	17	22	27	0	5	10	15	20	25	30	3	8	13	18
9	25	30	3	8	13	18	23	28	1	6	11	16	21	26	31	4	9	14	19
10	26	31	4	9	14	19	24	29	2	7	12	17	22	27	0	5	10	15	20
11	27	0	5	10	15	20	25	30	3	8	13	18	23	28	1	6	11	16	21
12	28	1	6	11	16	21	26	31	4	9	14	19	24	29	2	7	12	17	22

最大程度提高多核单次计算的L2命中率满足CubeBound需求；同时降低对L2 Cache大小的依赖。

CANN

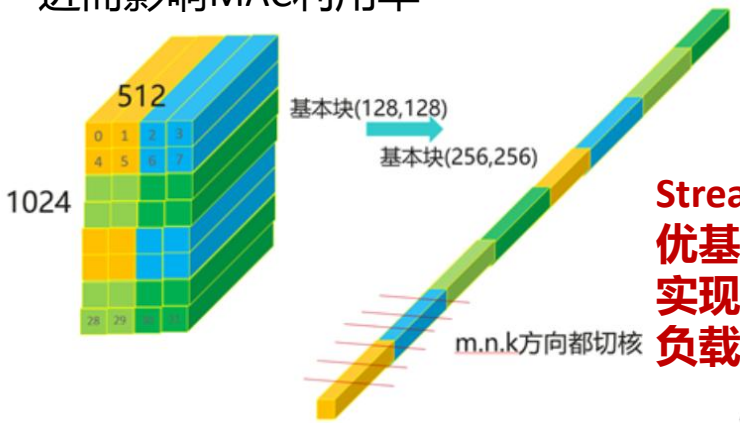
StreamK Tiling技术

传统IterateK的问题



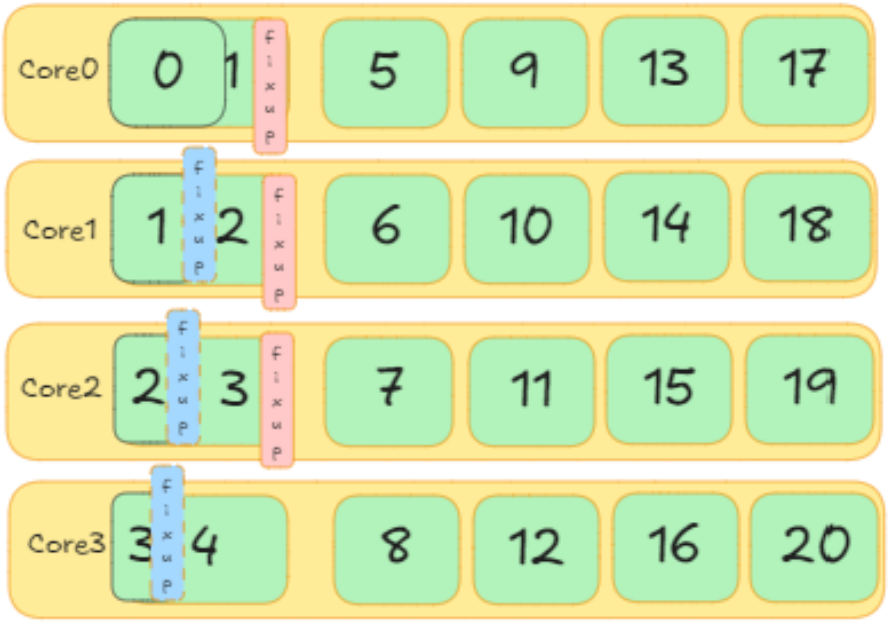
尾轮拖尾

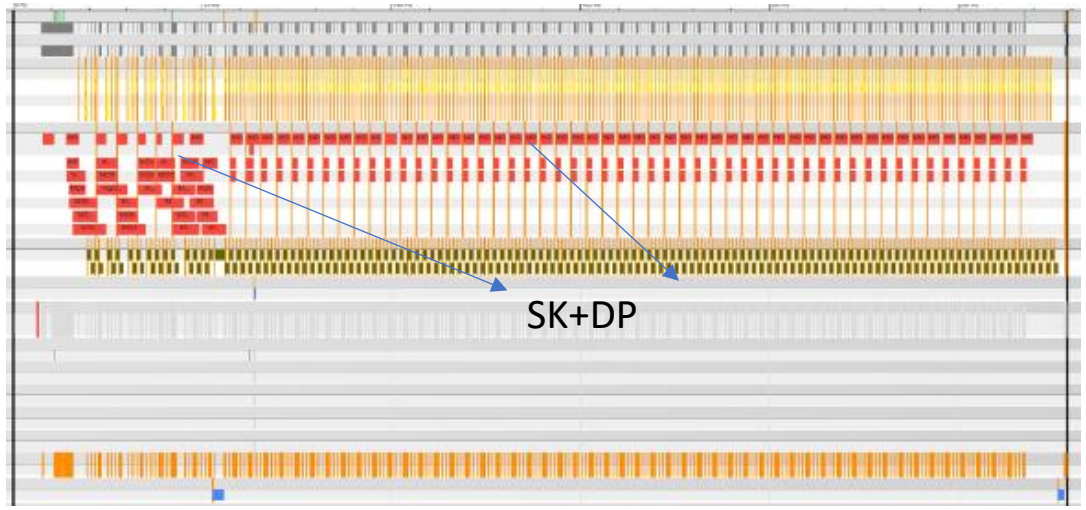
基本块减小导致带宽供应不足，
进而影响MAC利用率



StreamK在保持最优基本块的同时，实现多核之间的负载均衡

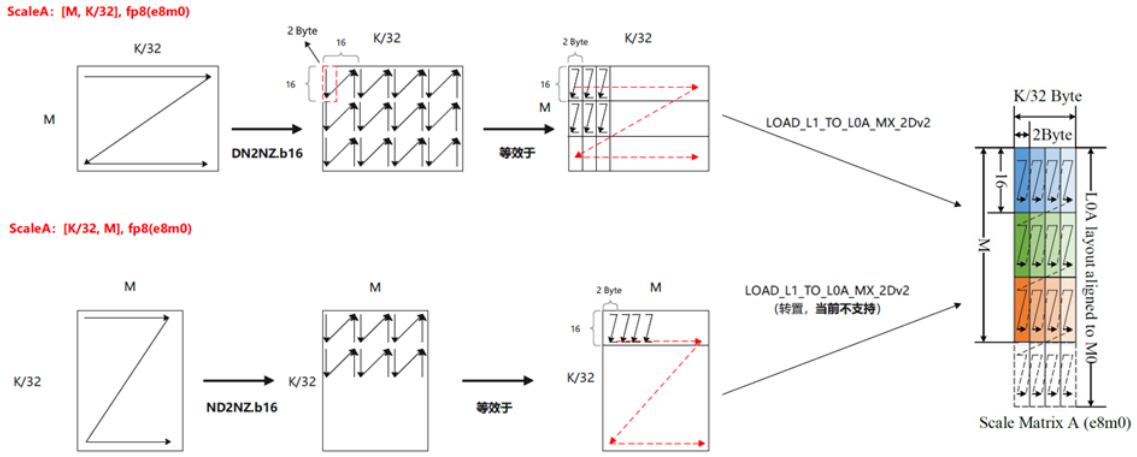
StreamK的解法



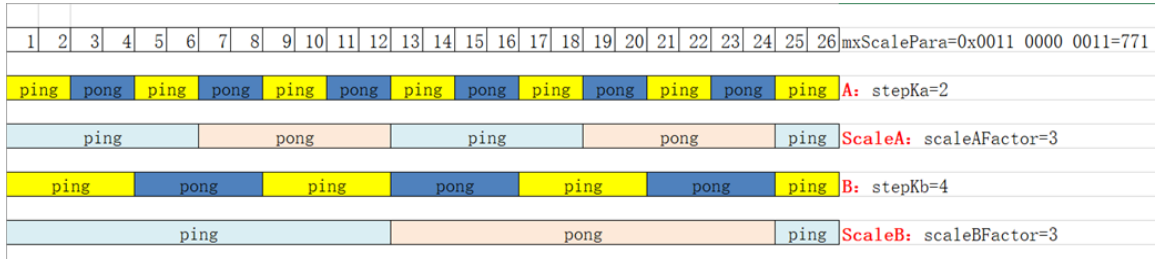


量化场景Kernel优化技术

MXFP: mxscale缓存技术

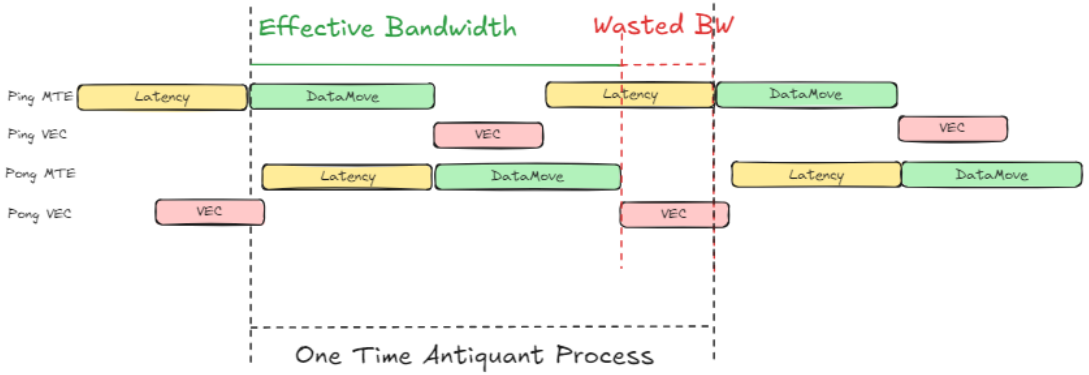


MXFP数据类型，32个Data共享1个Scale，如果Scale采用与Data相同的StepK会导致Scale的TileSize过小，导致带宽利用率较低，可以采用独立的scaleFactor缓存Scale。

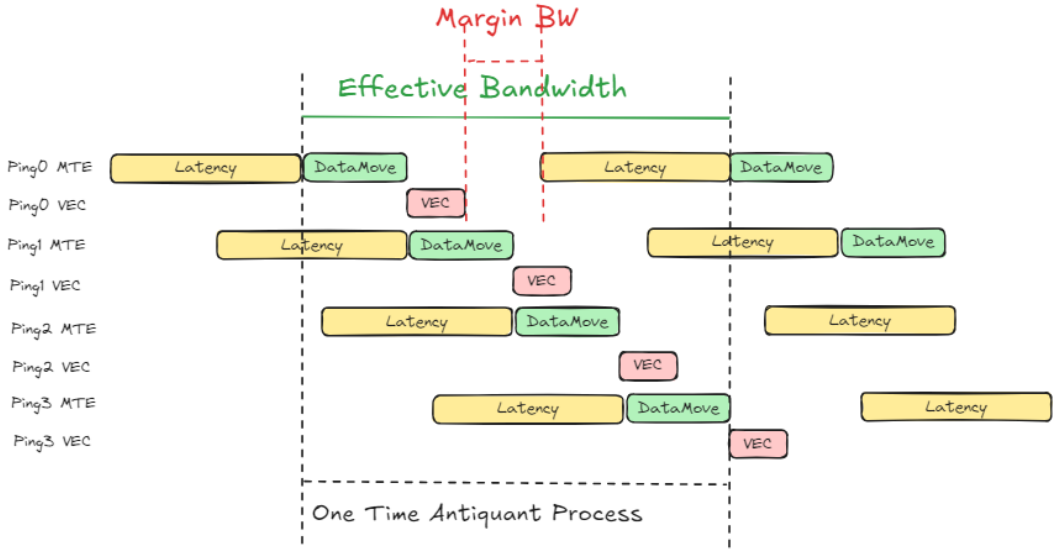


伪量化: n buffer技术

伪量化场景的antiquant阶段，使用Double Buffer，由于有效搬运不能掩盖Latency，所以存在Wasted BandWidth。



使用4 Buffer，一次的数据搬运Latency可以被三次的有效数据搬运掩盖。



目录

Part 1 Matmul性能极致优化

Part 2 RmsnormQuant性能极致优化

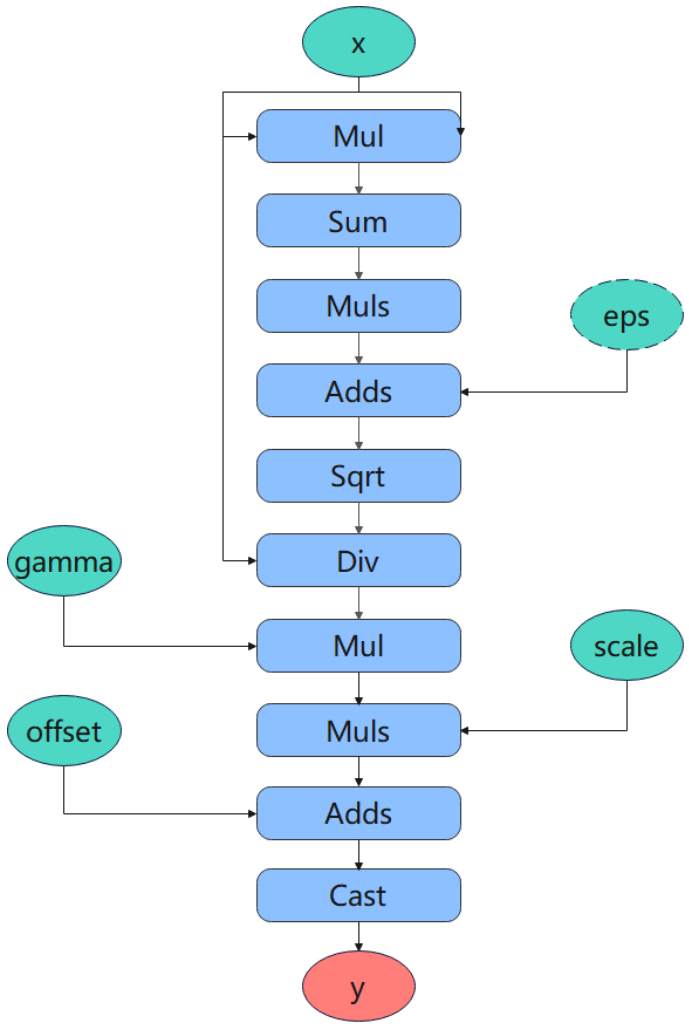
Part 3 FA类算子性能极致优化

Part 4 通信算子性能极致优化

Part 5 ops-samples仓介绍

RmsnormQuant算子功能介绍

计算图



功能介绍

- **简介：** RmsnormQuant算子是融合了均方根归一化（RMSNorm）与量化（Quantization）操作的高性能深度学习算子，主要应用于大语言模型（LLM）的推理场景。加速推理性能和降低内存占用。
- **参数：**
  - x：待处理的输入，shape为[..., D]。
  - gamma：可训练的缩放参数，shape与x的最后一维一致，即[D]。
  - eps：归一化计算预防除0的阈值，默认值为1e-6
  - scale：离线预设的量化缩放因子，shape与x的最后一维一致[D]或者为单点[1]。
  - offset：量化计算中的偏移量，shape与scale一致。
  - y：对x做均方根归一化后的量化结果，shape与x相同，数据类型为Hif8。

计算公式

$$y = Quant(RmsNorm(x))$$

$$RmsNorm(x_i) = \frac{x_i}{Rms[x]} * \gamma, \qquad Rms[x] = \sqrt{\left(\frac{1}{n}\sum_{j=1}^{n} x_j^2\right) + \varepsilon}$$

$$Quant(r) = Hif8(r * scale + offset)$$

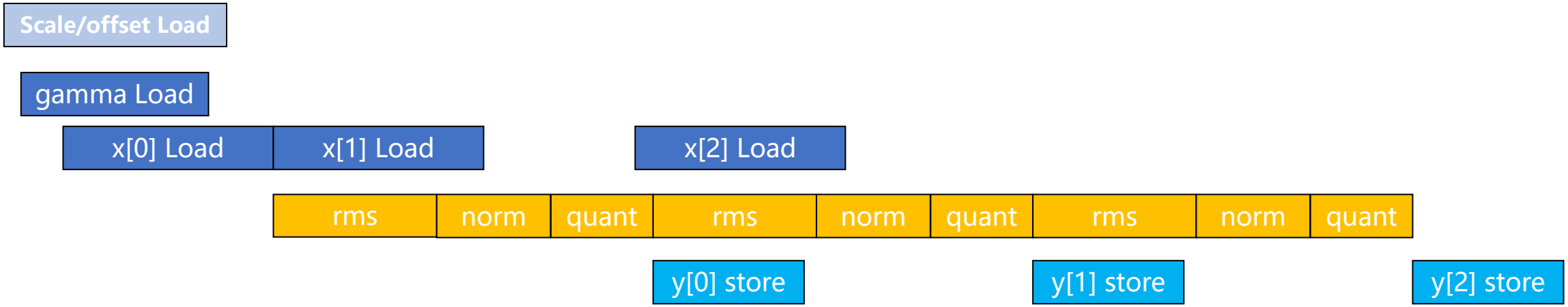
CANN

流水优化实现计算与搬运掩盖

流水优化

- **Double Buffer**：对输入x使用双缓冲buffer。
- **Gamma流水优化**：gamma的搬入仅在第一次搬入，后续缓存。
- **Scale和offset流水优化**：scale和offset在启动时搬入，避免中间过程打断流水。

流水图



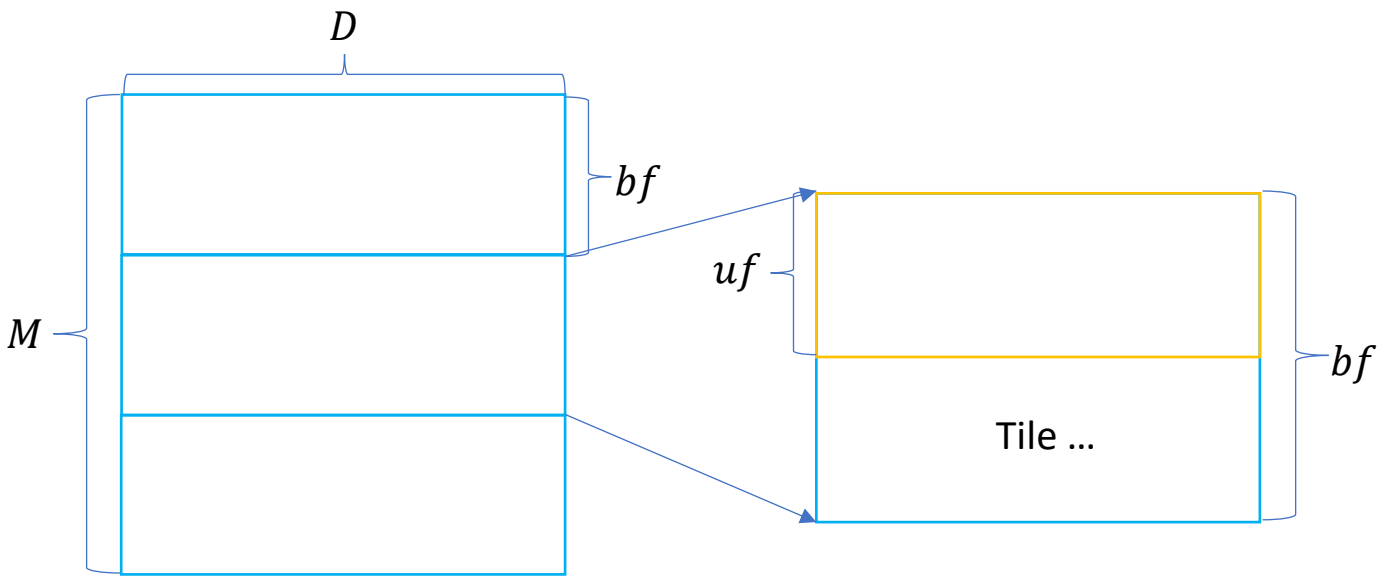
CANN

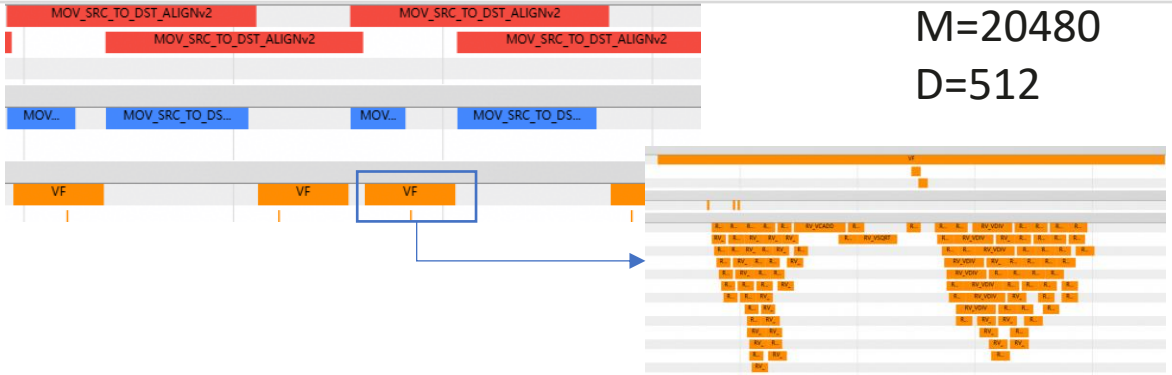
高性能Tiling实现

Tiling方案

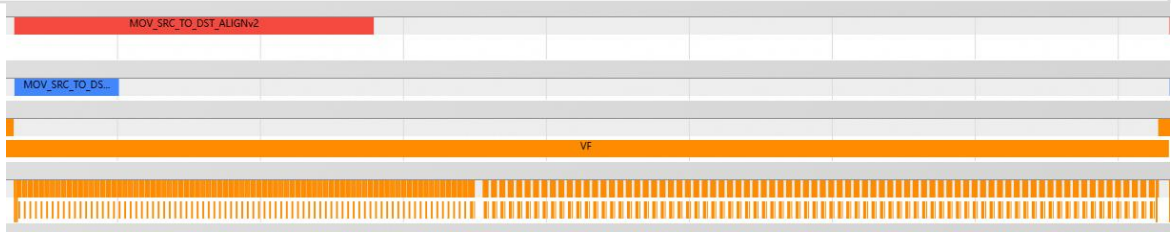
- **多核Tiling：**非尾轴合并（M轴）之后开多核，将多核在这个维度均分。
- **核内Tiling：**基于regbase特点划分buffer，按照用满UB的方式计算。

xBuf (x输入)	$Align(D) * uf$
rmsBuf (norm结果)	$Align(D) * Block$
gammaBuf (γ输入)	$Align(D)$
yBuf (输出)	$Align(D) * uf$
栈 (scale, offset输入)	8Byte





M=20480
D=512



单行切分（1个D）：86us　　IPC：0.48

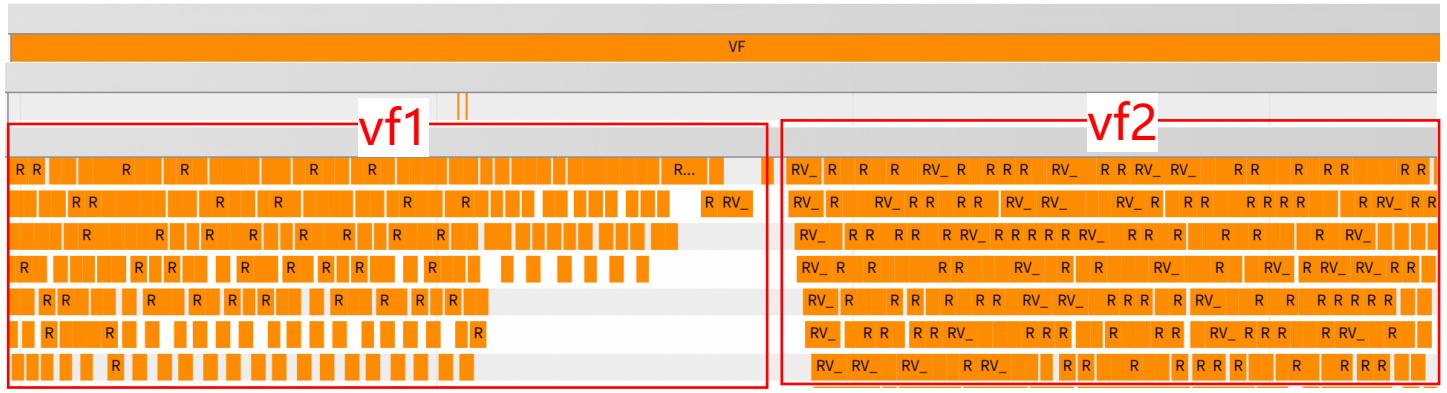
多行切分（多个D）：27us　　IPC：1.47

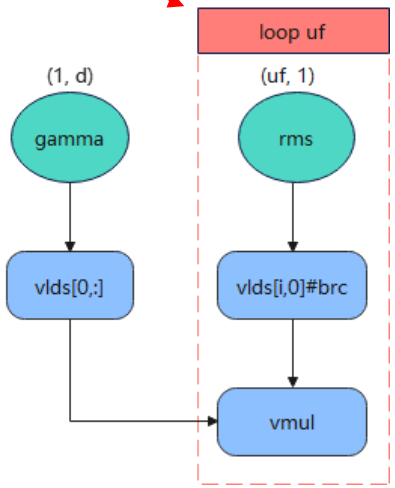
提升VF计算效率

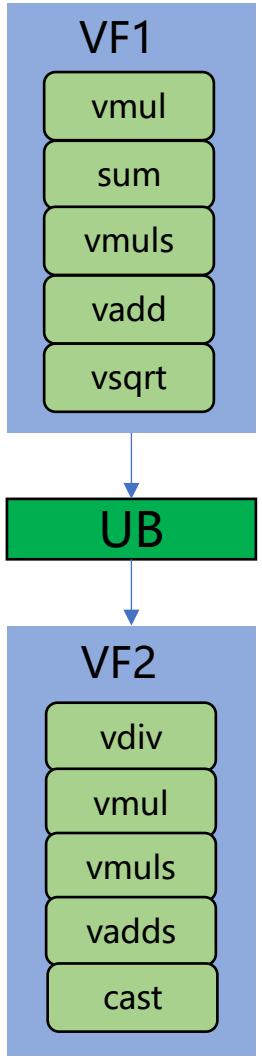
VF性能优化

- **VF融合：**以reduce节点将计算拆分成两个vf执行；vf1：均方根计算(rms)，vf2：归一化+量化(norm+quant)。当前支持手写融合，毕昇编译器自动识别，FlashVF工具寻优。
- **循环展开：**核内循环按照2/4/8展开，本样例为2，可在实际应用中调整，根据计算复杂度选择。代码实现通过添加program标记。
- **随路Broadcast：**将计算后的rms值随路broadcast到vf中

流水图

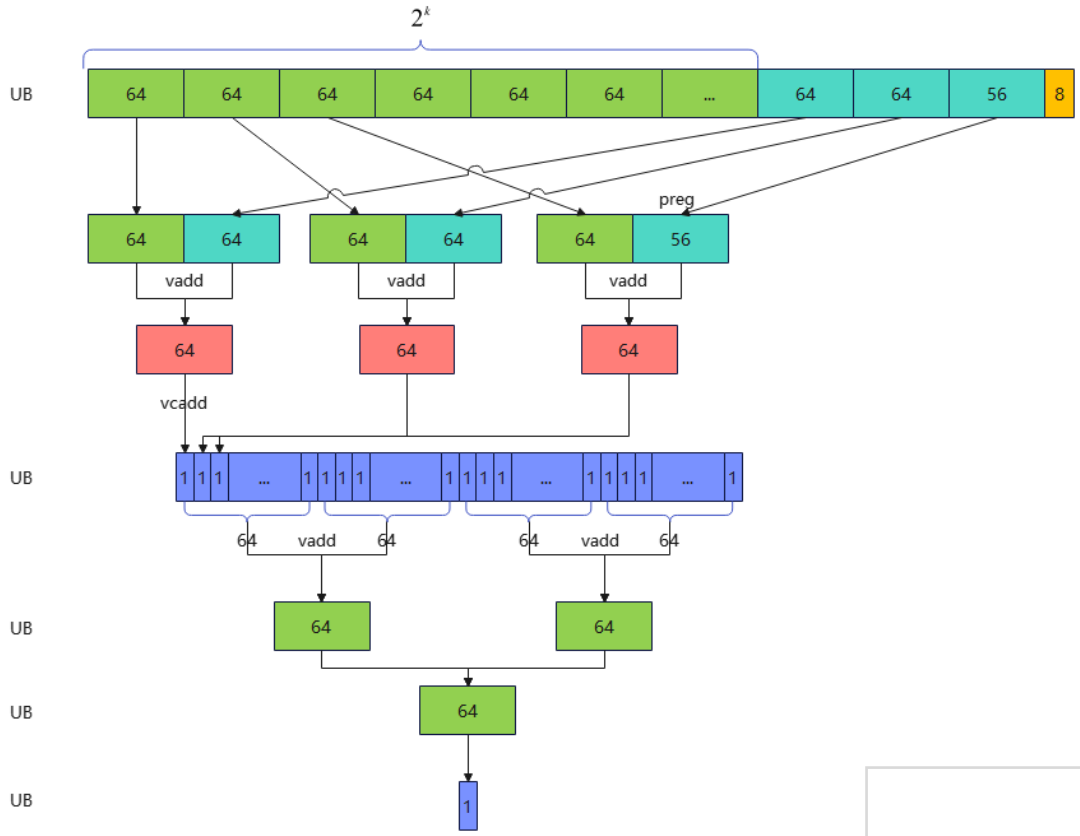






CANN

VF高效的二分累加计算



二分累加提升精度稳定性，
减少依赖发挥指令双发能力

核内二分累加

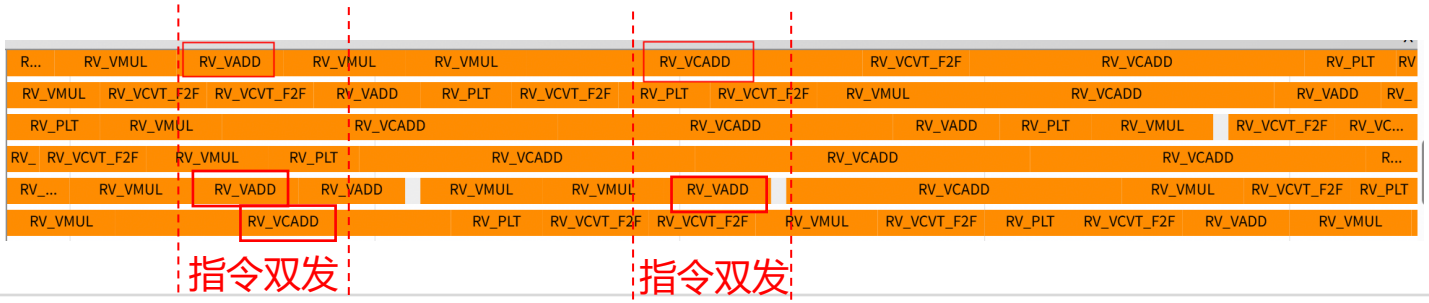
Step 0：数据按照2^k划分为整尾块，按照VL长度（64个fp32）将尾块部分与整块的头部相加，将带尾块的数据变成2^k，使用vadd指令，可支持指令双发；

Step 1：对无尾块的数据使用vcadd指令将数据reduce到单个值，结果写出到UB。数据已经少了至少64倍，极大降低对UB读写带宽的诉求。

Step 2：将写到UB的数据重新读入，每次读入$2 \times VL$个相加，计算的结果再写回UB

Step 3：重复Step 2，直到reduce结果是单个值为止

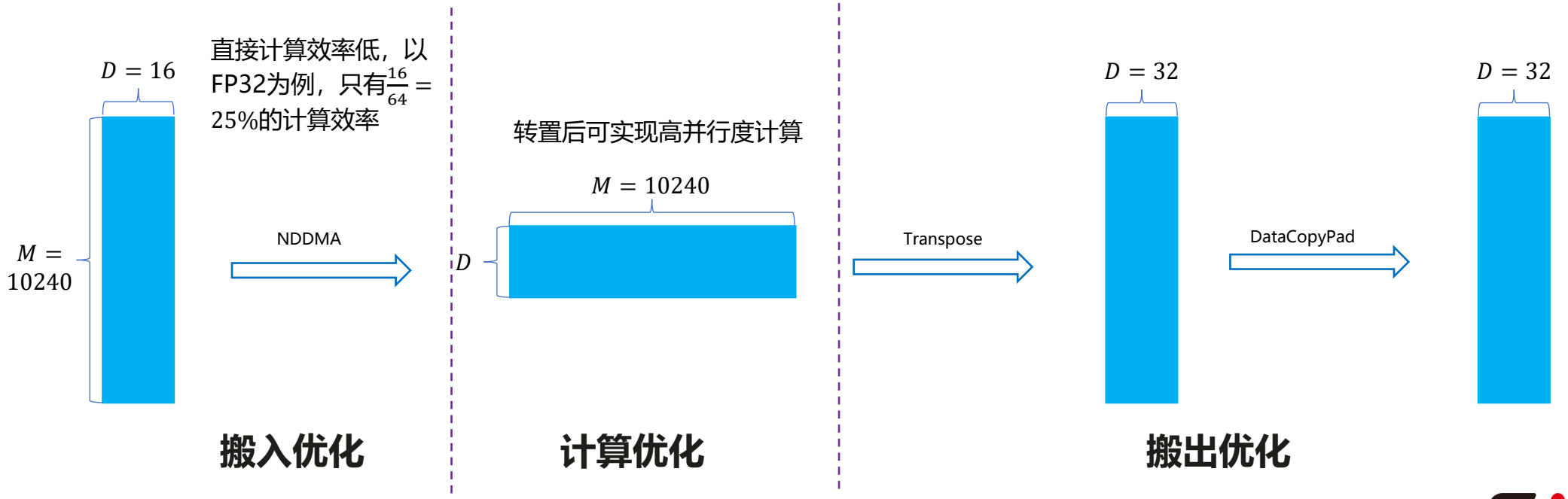
流水图



提升小包（小于128B）计算效率

小包计算效率优化

- **搬入优化：**将D轴通过NDDMA指令转置到高轴，为last轴凑足够大的并行度。
- **计算优化：**last轴reduce变成非last轴reduce。
- **搬出优化：**通过转置指令将D轴转到last轴，然后再搬出



目录

Part 1 Matmul性能极致优化

Part 2 RmsnormQuant性能极致优化

Part 3 FA类算子性能极致优化

Part 4 通信算子性能极致优化

Part 5 ops-samples仓介绍

举例：FIA算子简介

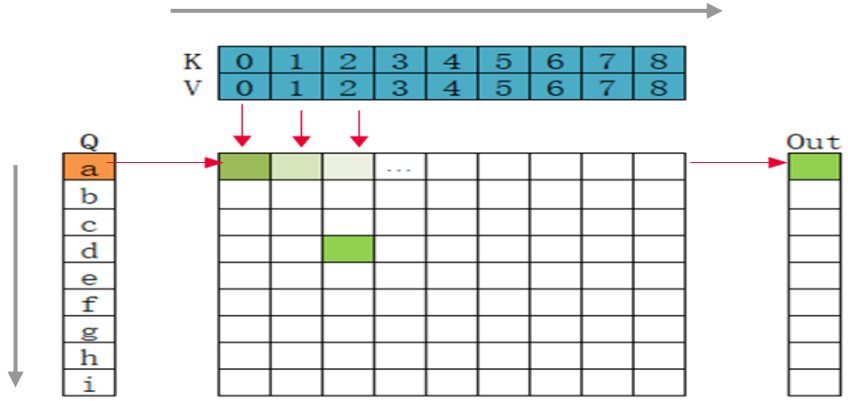
FIA（FusedInferAttentionScore）算子，是基于经典的Flash-Attention算法实现的融合算子

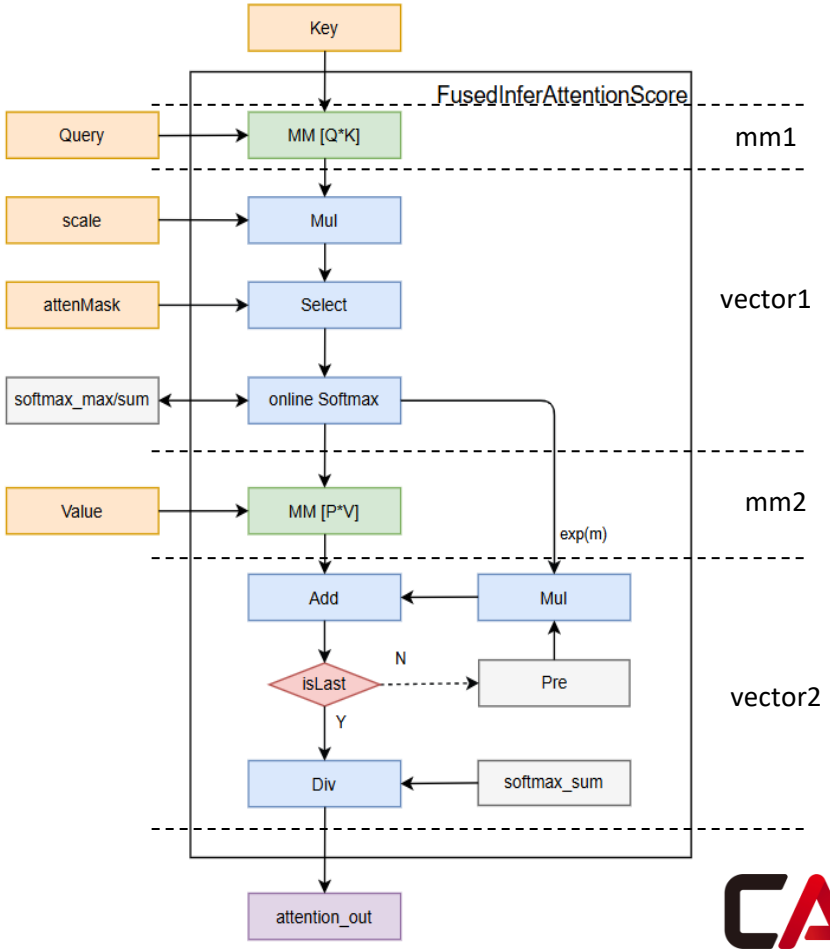
Q、K、V进行切块处理，对切块后的Q、K、V逐步计算；

切块后的Q、K做矩阵乘，进行局部softmax计算得到P，并在这个过程中，维护局部的归一化因子（最大值、累积和）；

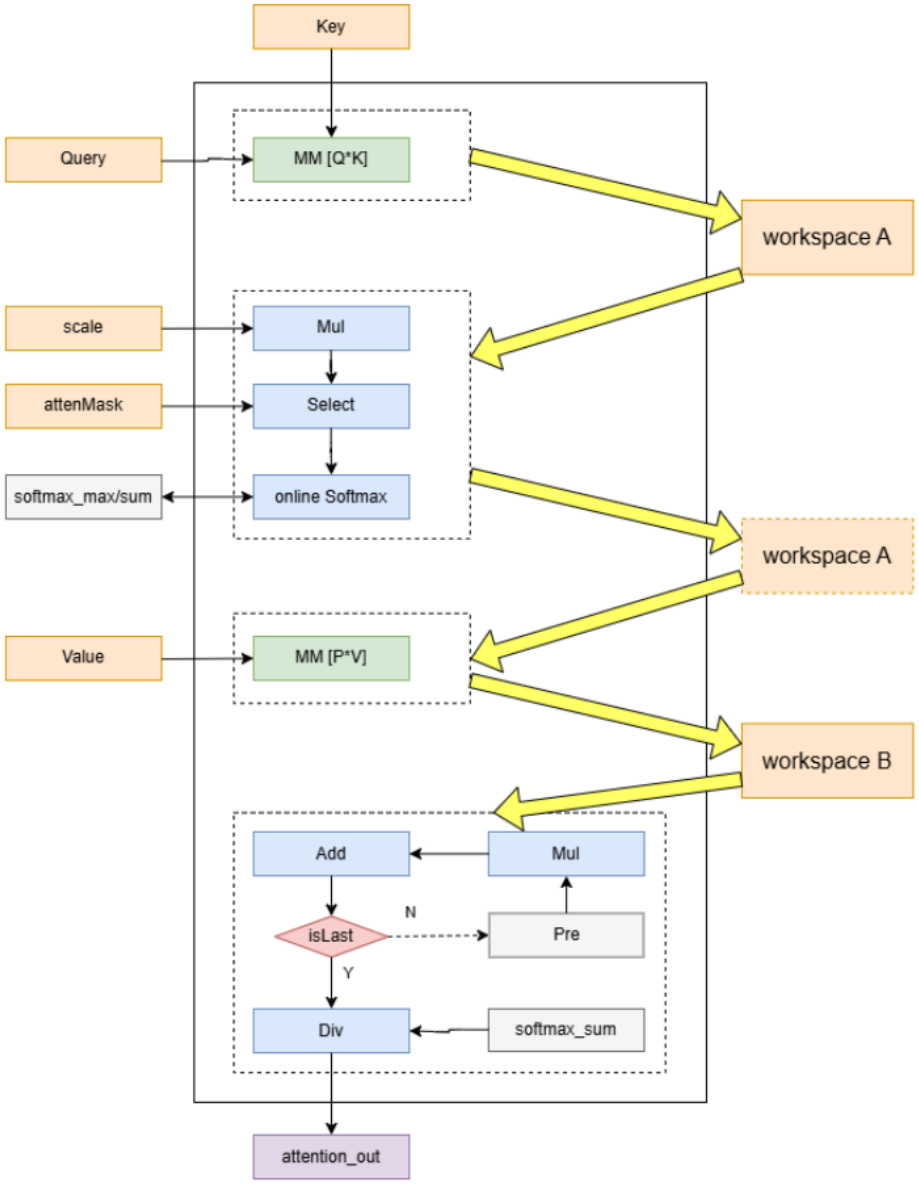
P与切块后的V做矩阵乘，得到的结果，累加更新到历史的PV结果中；

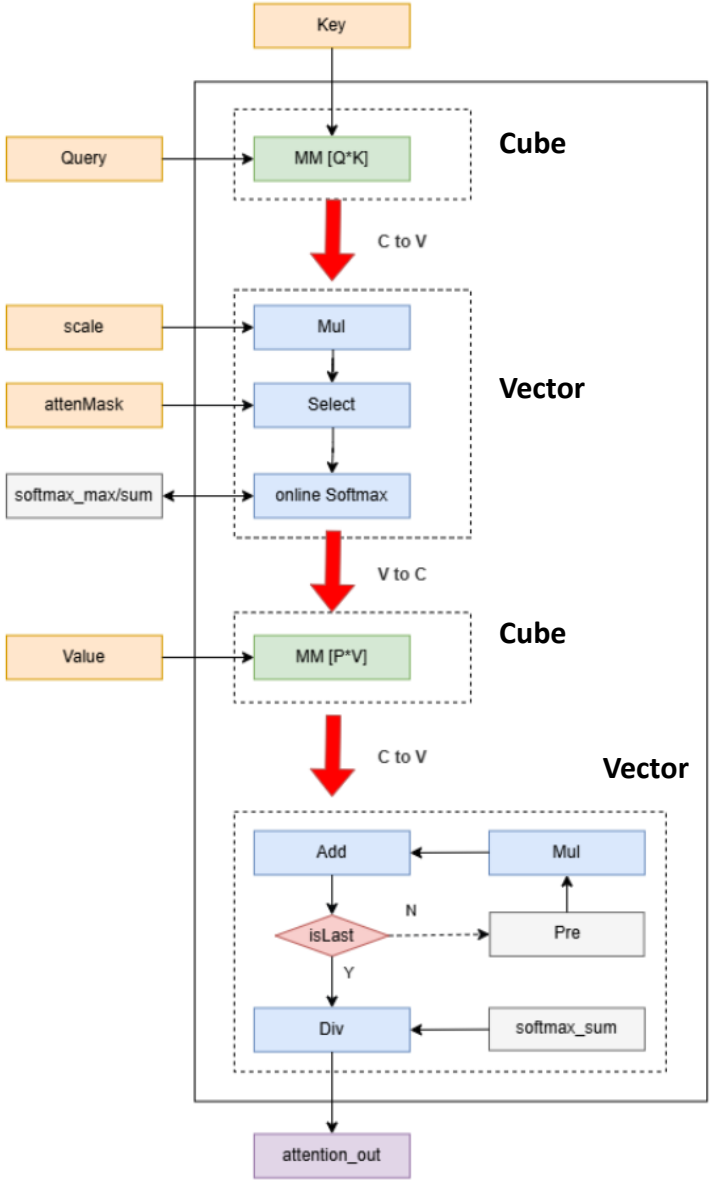
逐步计算，最后得到完整的Attention结果；





CV共享通道带来的性能提升





FIA在计算过程中，利用CV之间的通路

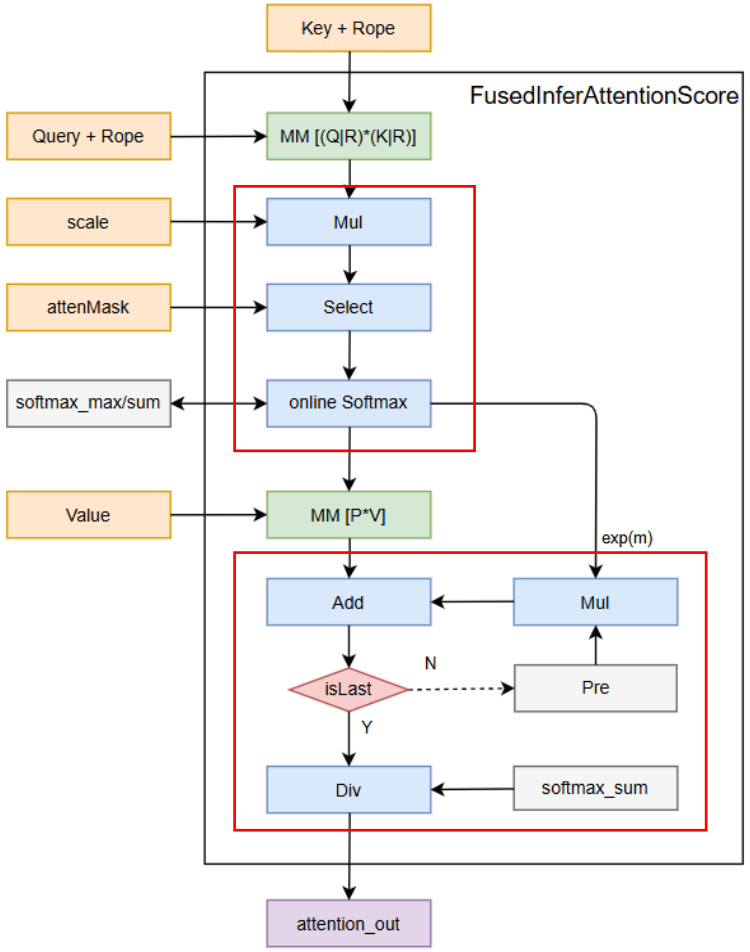
将Cube（Q*K P*V）计算结果直接存储到Vector的UB上；

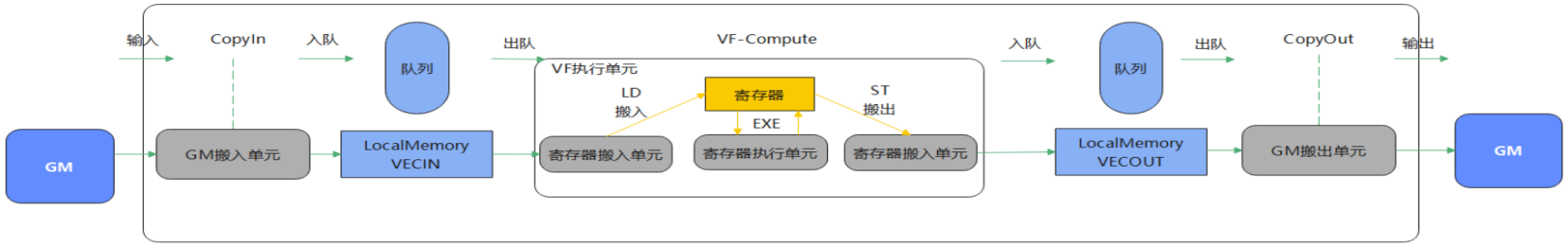
将Vector的计算结果直接存储到Cube的L1上；

避免中间数据进入到GM（workspace）而增加片上内存的访存量

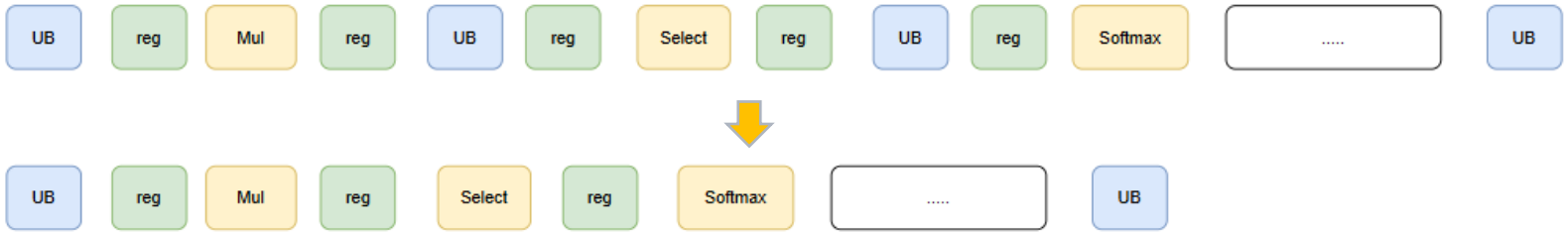
CANN

VF融合方案带来的性能提升

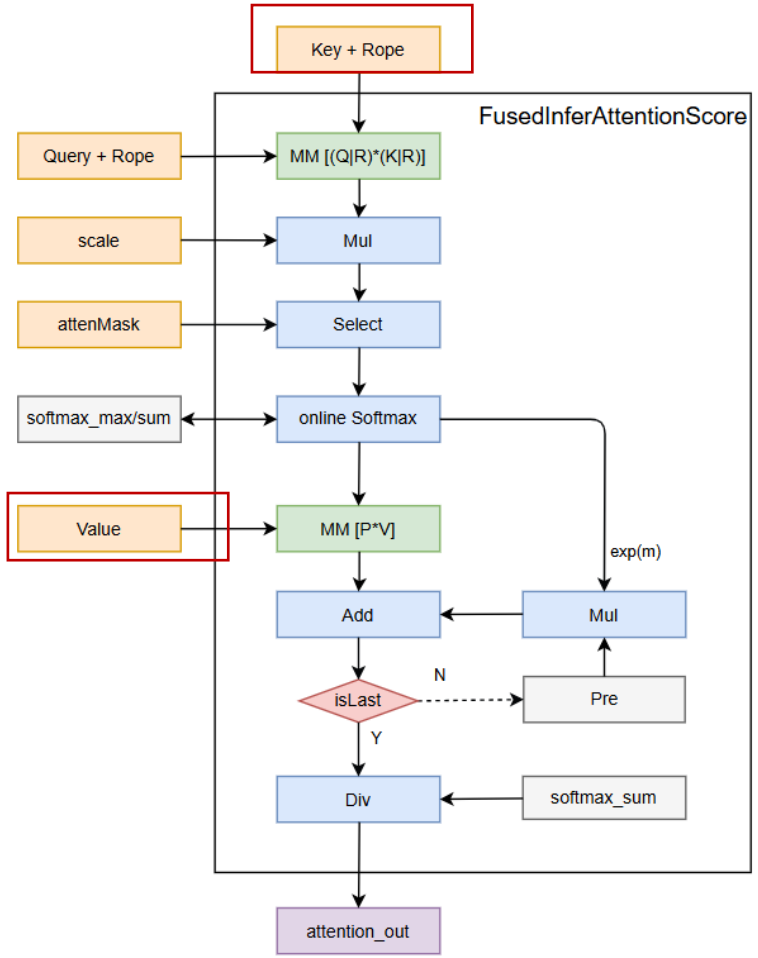




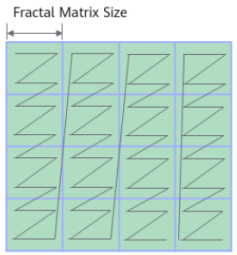
FIA在Vector1、Vector2计算过程中，利用VF融合编程技术，将Vector1中的Mul、Select、Softmax、Cast、Transdata(ND2NZ)计算融合成一个VF计算，将Vector2中的Add、Mul、Div计算融合成一个VF计算，避免计算过程中间数据LD/ST到UB中



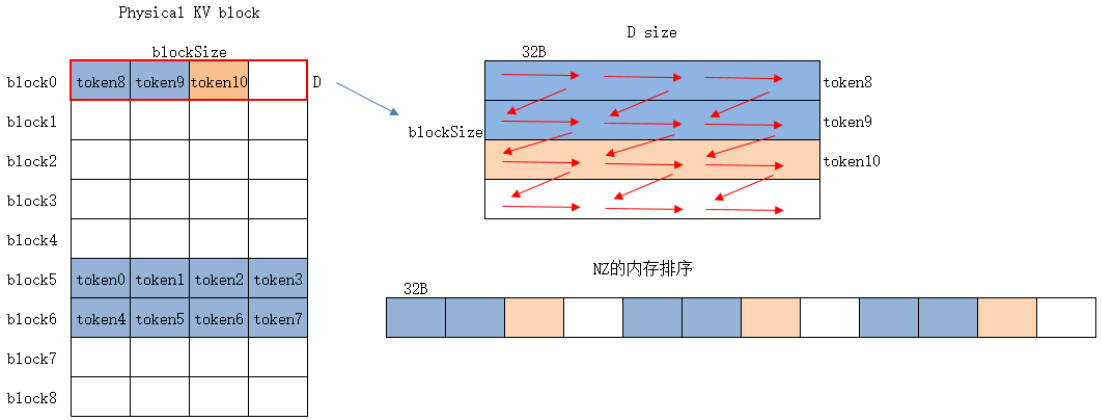
Layout排布优化



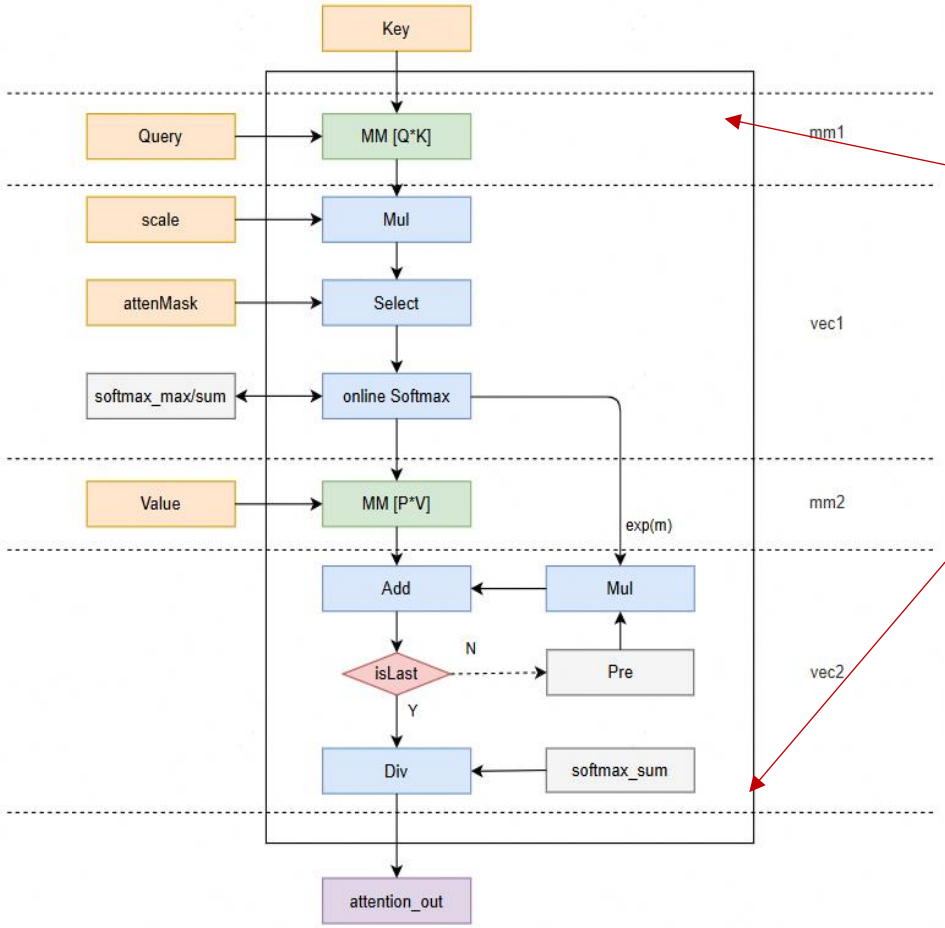
NZ 数据格式是一种专为提升计算性能而设计的特殊数据排布格式，在昇腾芯片中被广泛使用，能最大化挖掘 Cube 计算单元的性能潜力，在矩阵乘法操作中带来更高的计算效率

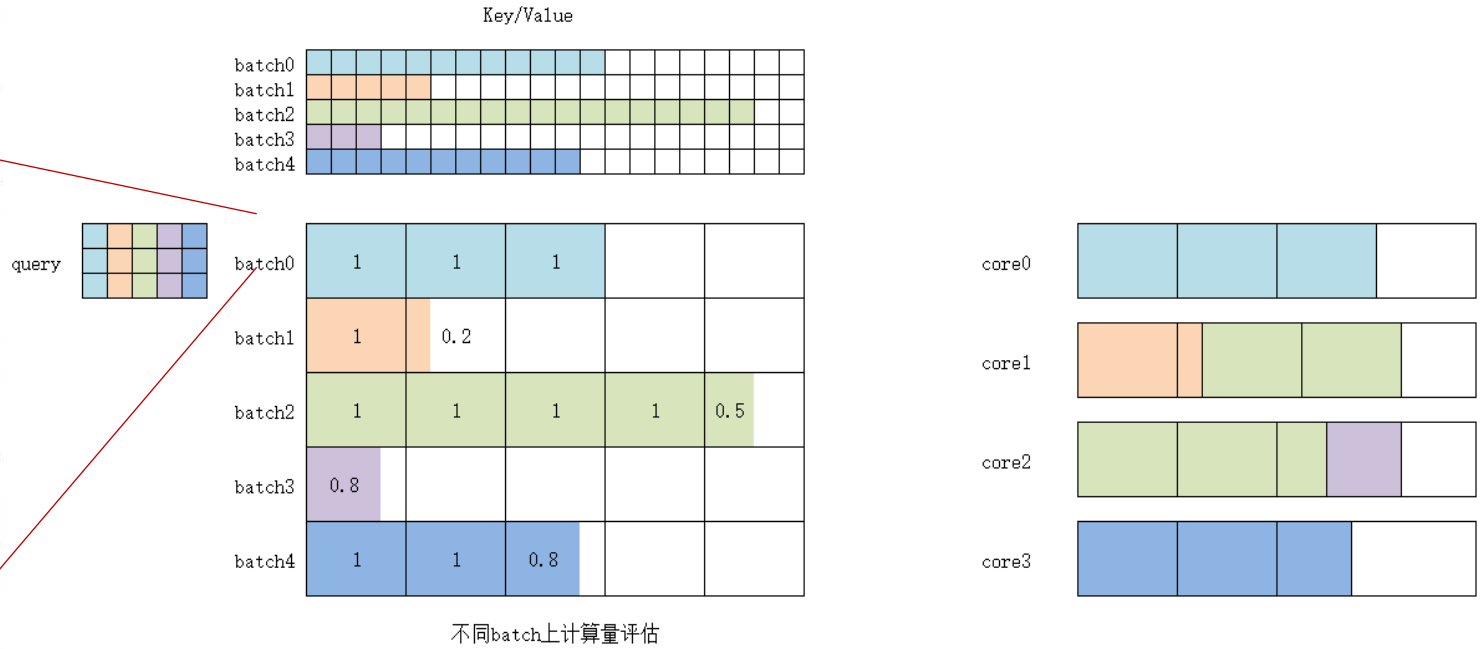


FIA算子直接按NZ格式将Key、Value数据从片上内存读入到Cube计算单元的内存中，避免访存过程中的ND、NZ数据格式转换，从而提升FIA算子的访存效率



更优的负载均衡优化方案





不同Batch的计算耗时不相同。FIA根据实际Key/Value数据量大小（actual-sequence），对每个Batch的计算量（耗时）进行评估。

按Tiling块粒度，将每个Tiling块的计算均摊到多个核上进行计算。

并且支持利用Flash-Decoding技术，将Key/Value切分到不同核上进行计算。

目录

Part 1 Matmul性能极致优化

Part 2 RmsnormQuant性能极致优化

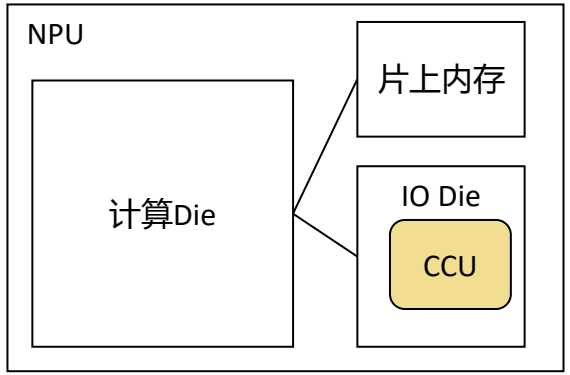
Part 3 FA类算子性能极致优化

Part 4 通信算子性能极致优化

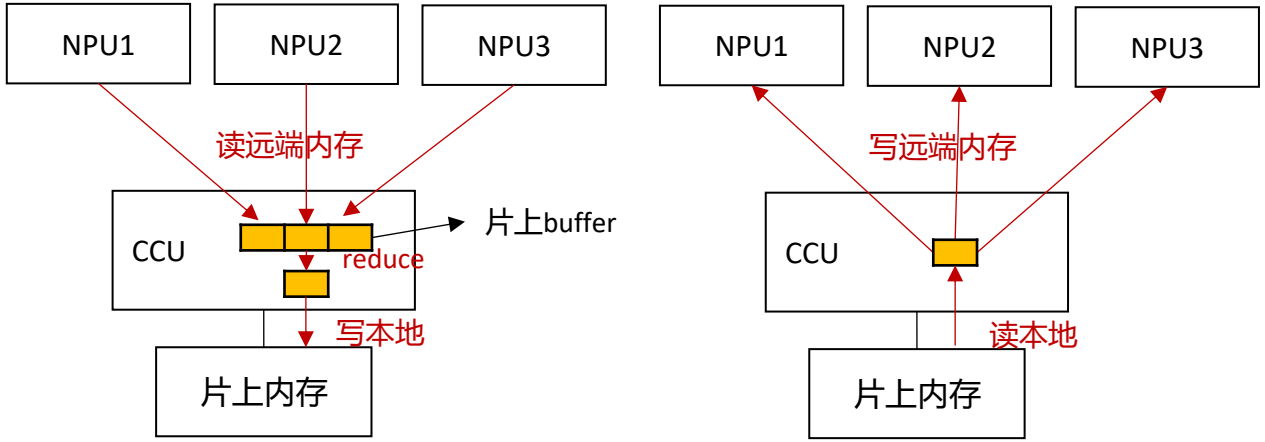
Part 5 ops-samples仓介绍

专用通信引擎CCU – 降低内存带宽占用、降低通信时延、天然零拷贝及确定性计算

Collective Communication Unit（CCU）是Ascend 950上新增的用于加速集合通信的专用处理器，CCU根据软件预置的集合通信算法指令，完成通信节点间的同步、地址交换、数据传输以及规约计算，实现集合通信操作



CCU位置示意图



CCU 核心能力

解决的问题：

1、利用CCU片上buffer缓存通信数据，减少访存带宽

2、利用片上buffer+计算单元执行Reduce操作，确保计算保序

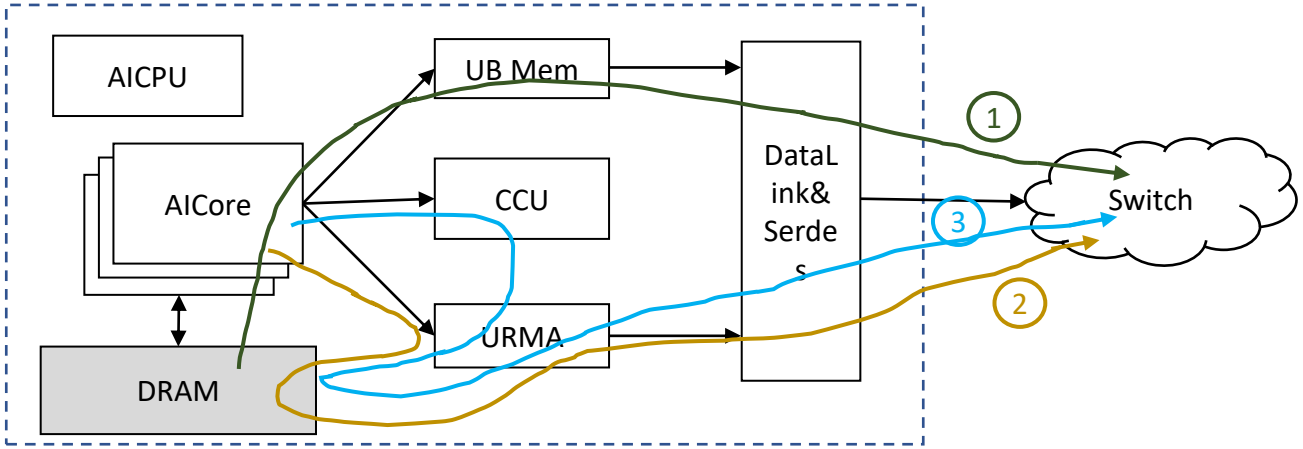
3、专用的通信调度与同步机制，降低时延，释放计算算力

4、消减用户buffer到CCL Buffer的拷贝

CANN

通算融合算子开发方式

950的通信方式



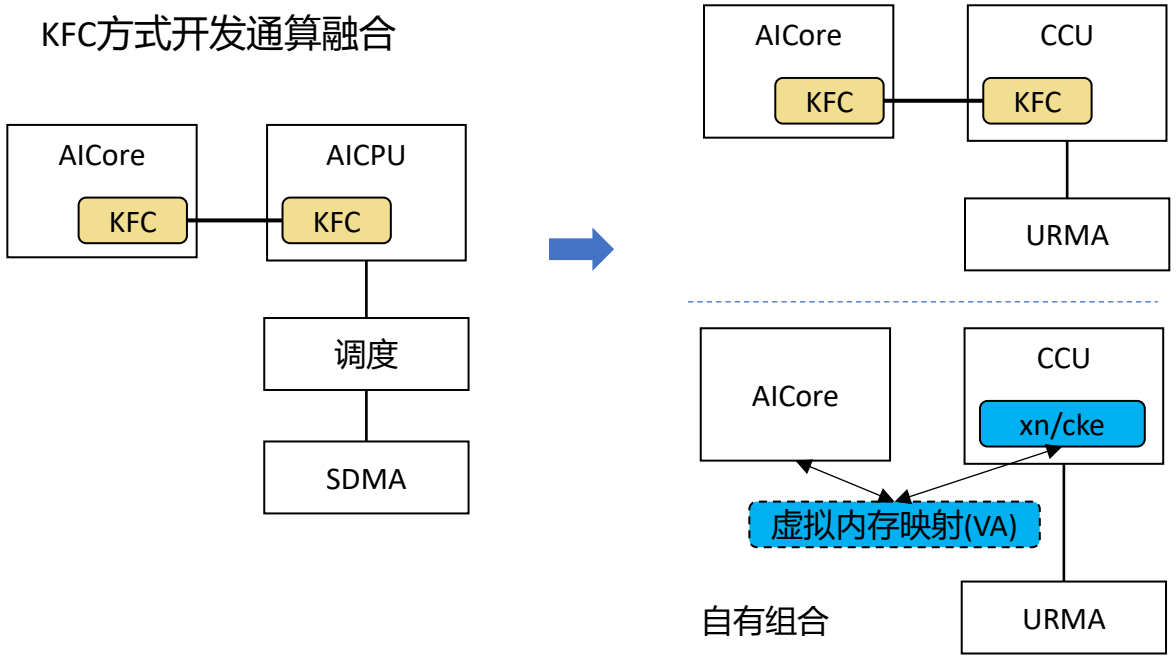
通算融合开发形式：

1、使用AIV + UBMem的通信方式

2、使用AIV直驱URMA通信方式

3、使用AIV + CCU的通信方式

KFC方式开发通算融合



解决的问题：

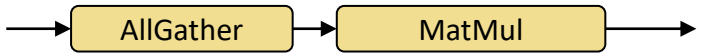
1、 使用CCU进行通信，不占用计算核

2、 使用CCU进行通信较原来AICPU，通信任务展开更快从AICore中发起任务通信到通信操作开发的静态时延小。

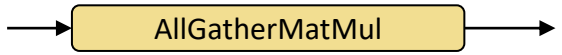
CANN

AllGatherMatMul的极致性能优化

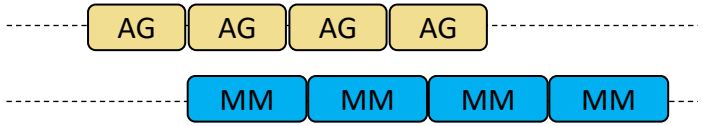
算子功能



将AllGather+MatMul融合成一个算子



算子kernel内，对数据进行分块，将多块间的通信和计算流水并行起来

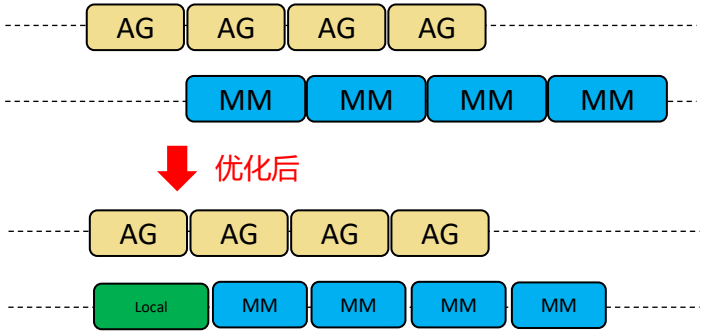


优化1：本地块计算提前

优化收益点：

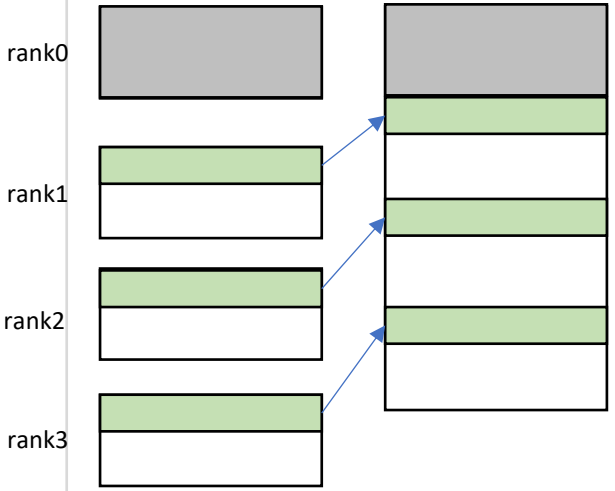
本地块计算提前计算，相当于将MM的权重预取了一遍，后续的MM计算更加高效

本地块提前计算，后面计算的总数据量减少，计算更快



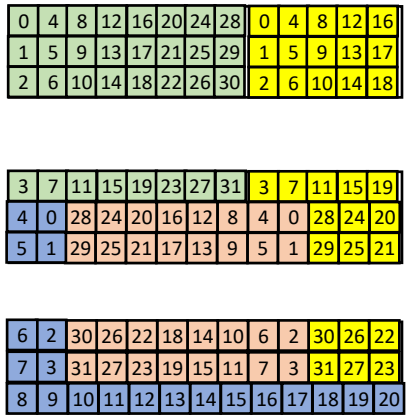
★ 模型Case掩盖率达70%

优化2：非连续块计算



从各个卡上Gather回来的数据在空间分布上会一定的非连续。

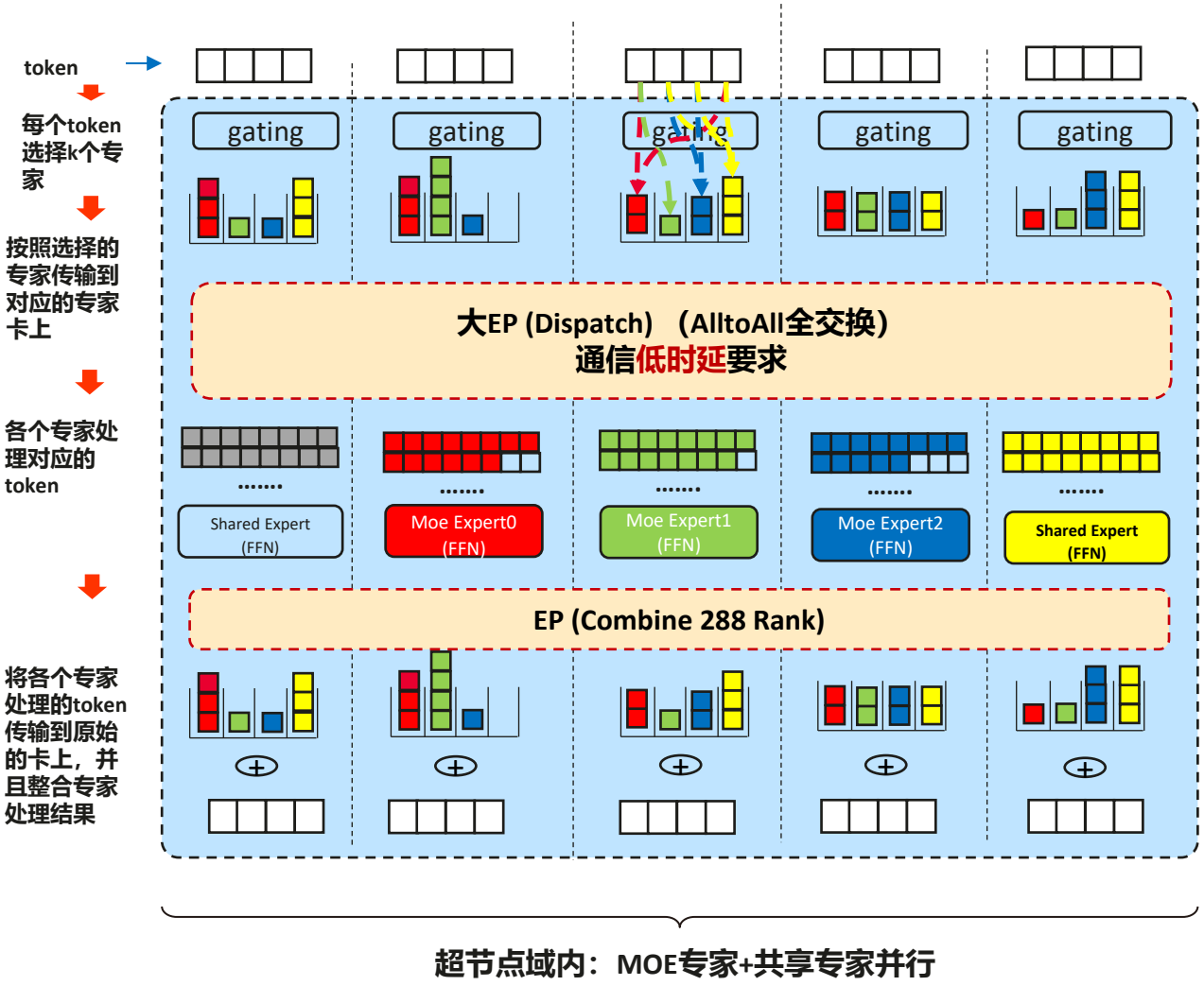
将非连续看成连续的进行SWAT方式的MatMul分核计算，保证MatMul不会出现劣化。



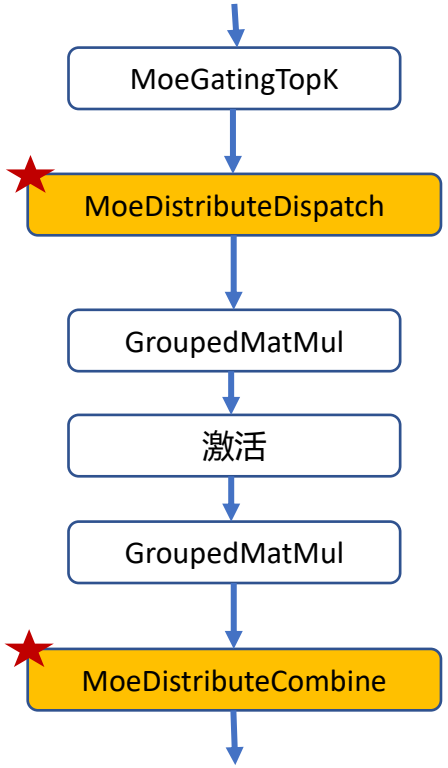
Dispatch & Combine的极致性能优化

算子功能

随着MOE大模型的发生，专家并行（EP）规模的不断扩大，专家间的token分发和combine成为大模型推理的性能提升关键点。MoeDistributeDispatch与MoeDistributeCombine即为完成该场景高效通信而设计的融合算子



实现融合算子优化通信性能

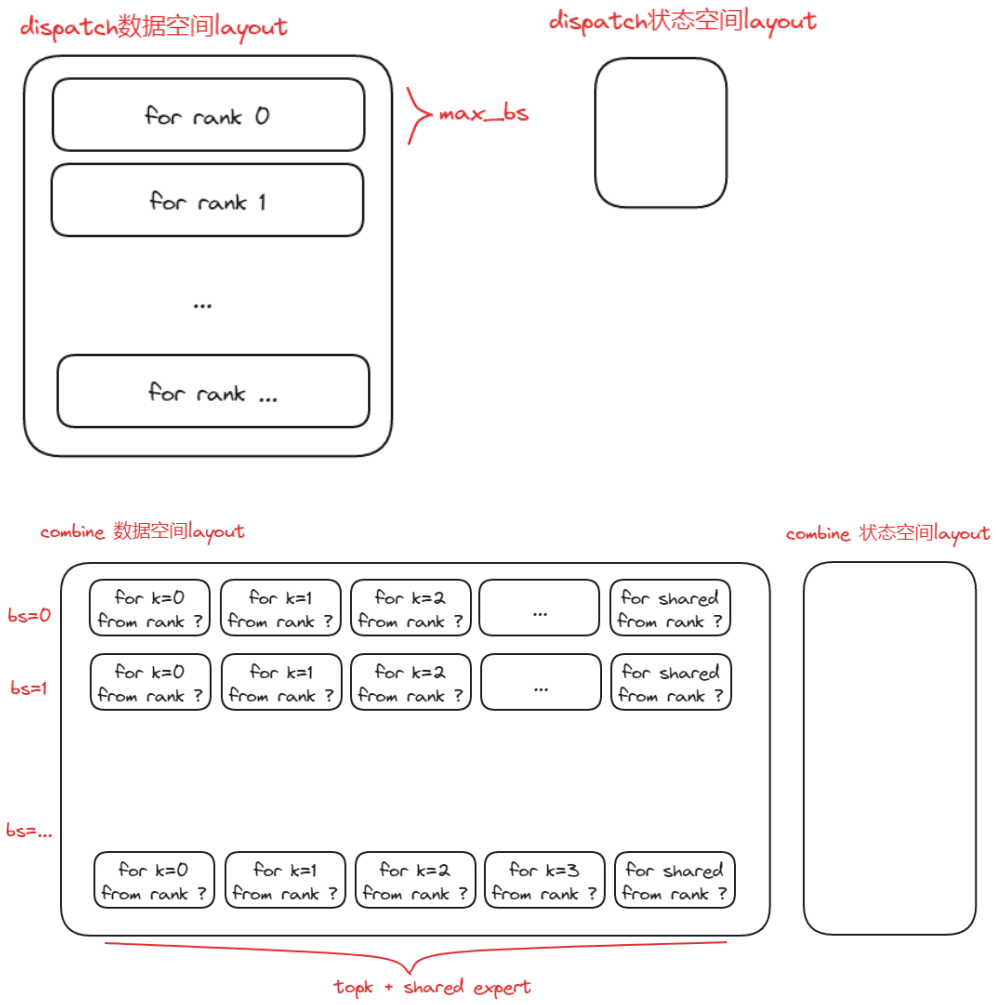


CANN

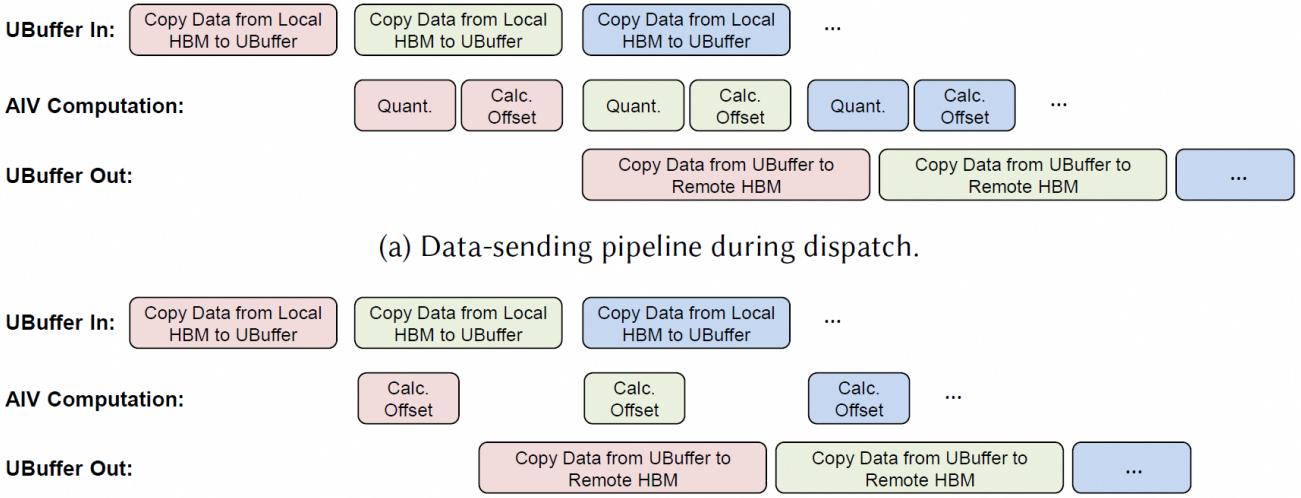
Dispatch & Combine的极致性能优化

高性能实现关键措施

✓ 双buffer空间，特定layout排布支撑单边写通信，消除常规通信的前同步



✓ Dispatch量化前置，量化与通信流水并行，减少通信数据量，掩盖量化开销。



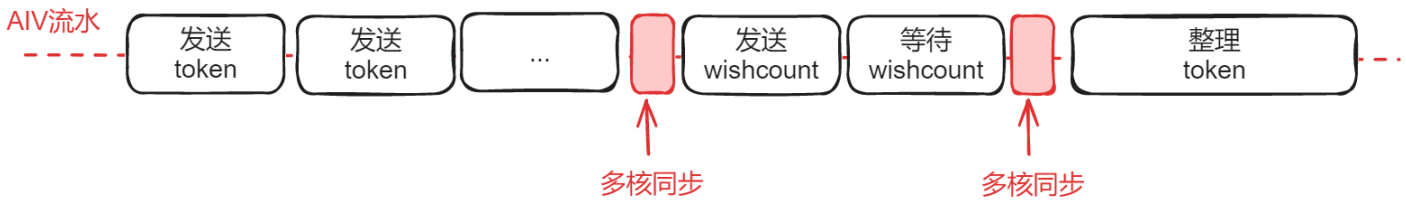
(a) Data-sending pipeline during dispatch.

CANN

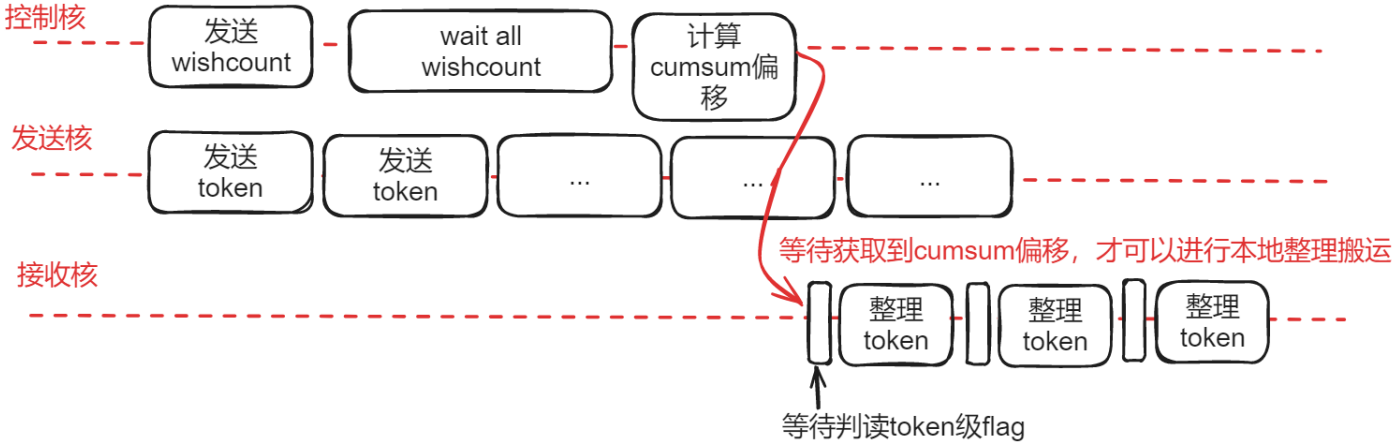
Dispatch & Combine的极致性能优化

进一步极致性能优化： 20+%收益

根据我们芯片512B包写的颗粒度大小，在其中设置对应的标识位，标识和数据原子写



优化，消除多核同步行为，同时可以间本地token整理和token发送的通信进行一定的overlap掩盖



CANN

目录

Part 1 Matmul性能极致优化

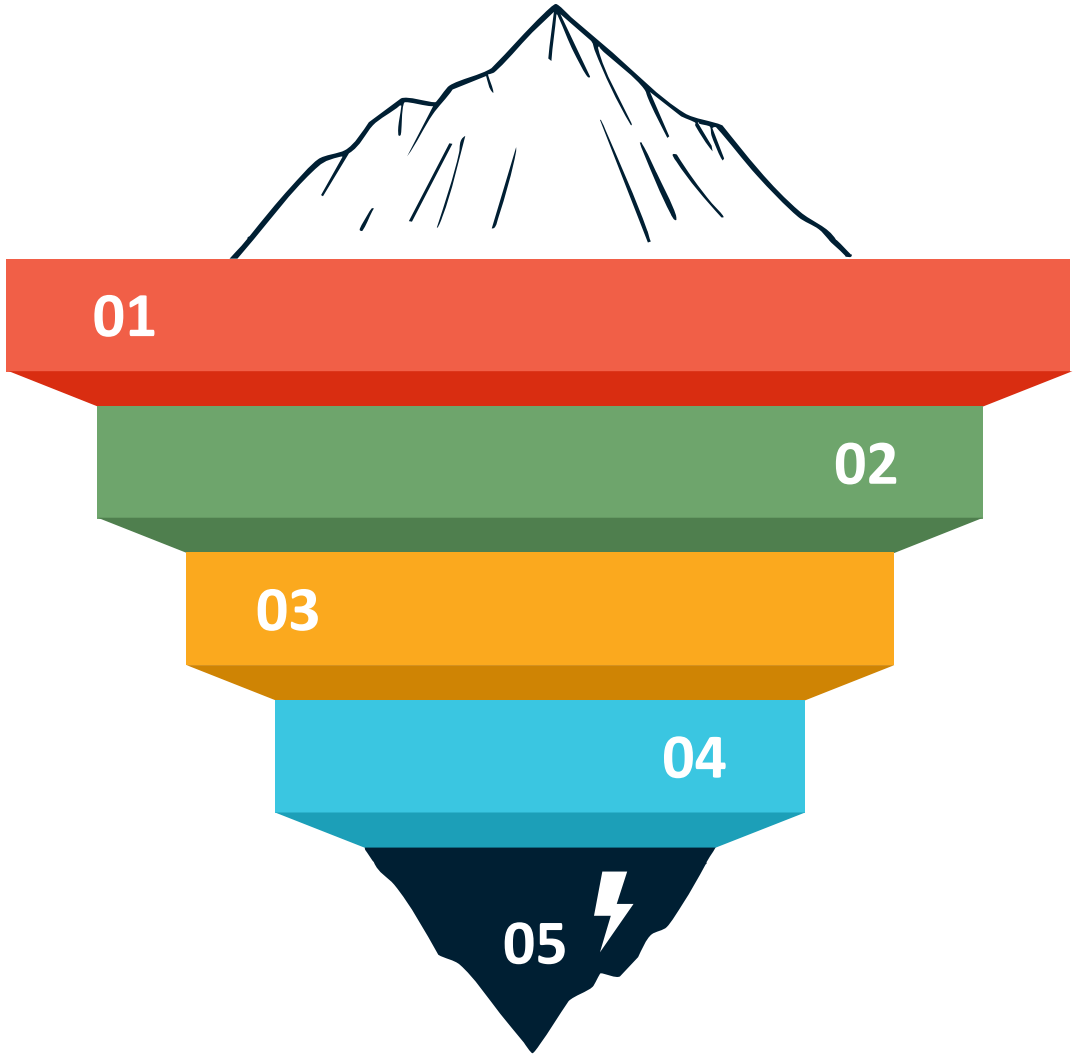
Part 2 RmsnormQuant性能极致优化

Part 3 FA类算子性能极致优化

Part 4 通信算子性能极致优化

Part 5 ops-samples仓介绍

算子开发的冰山模型



从功能跑通进阶到极致性能，才真正踏入了算子开发的**深水区**
越向深处，挑战越大

01 功能跑通	算法逻辑正确 / 编译通过
02 资源利用	多核并行 / Tiling设计 / 负载均衡 / 流水并行
03 访存优化	合并访存 / UB缓存 / 对齐访问 / 数据预取 / L2 Cache
04 指令调优	寄存器规划 / 指令精简 / 循环展开 / VF融合
05 系统优化	计算Tiling / Kernel启动开销优化

https://gitcode.com/cann

CANN

ops-samples简介 https://gitcode.com/cann/ops-samples

背景和目标

背景

1. 算子开发门槛不高，高性能算子开发门槛高，开发者性能调优有难度。
2. 大模型关键算子复杂，优化手段多。
3. 关键算子有深度定制需求，追求卓越性能。

目标

1. 提供算子领域高性能实操演进样例与体系化调优知识库。
2. 让开发者系统性掌握从逻辑实现到精细化调优的演进路径。
3. 提供关键算子的优化细节。

https://gitcode.com/cann

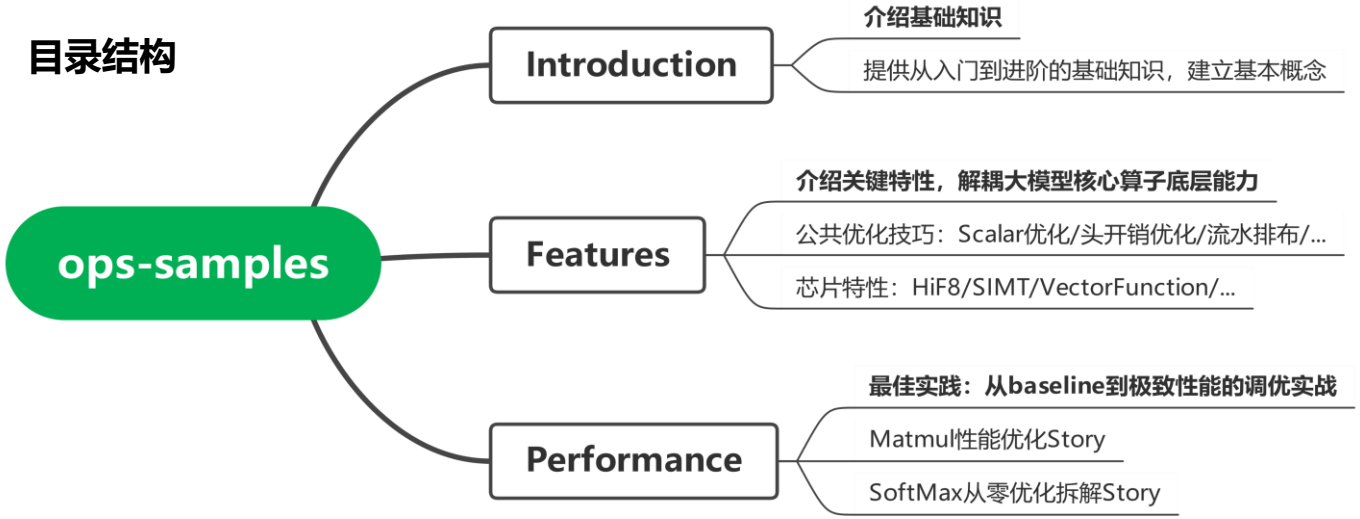
代码仓全景

三大板块

1. Introduction: 介绍基础知识，建立基本概念，补全从入门到精通的知识空缺。
2. Features: 介绍关键特性，解耦大模型核心算子底层能力。包括公共优化技巧和关键芯片特性。
3. Performance：最佳实践，从baseline到极致性能的调优实践。包含中间态代码，支持独立调试学习。

针对冰山模型的深水区挑战，ops-samples 将隐性的调优经验转化为显性的代码资产，构建了阶梯式的成长路径。

目录结构



CANN

ops-samples: 1-100分的进阶指导

全景入门教程

入门教程

samples (构建中)

为CANN初学者提供0->1的系统化学习路径、快速入门以及参考实现。

组件功能示例

集中样例仓

ops-samples

提供算子领域高性能实操演进样例与体系化调优知识库，让开发者系统性掌握从逻辑实现到精细化调优的演进路径。

组件examples目录

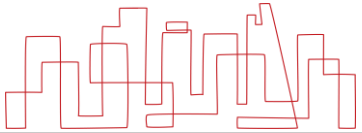
ops-math/examples

ops-nn/examples

ops-cv/examples

ops-transformer/examples

让开发者快速上手的简易调用样例，



行业落地案例库

Recipes系列

cann-recipes-infer

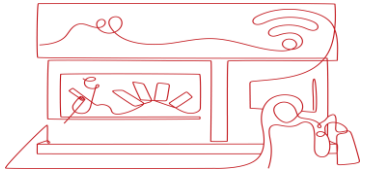
cann-recipes-train

cann-recipes-embodied-intelligence

cann-recipes-spatial-intelligence

cann-recipes-harmony-infer

模型落地最佳实现，提供典型模型的优化样例



https://gitcode.com/cann

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

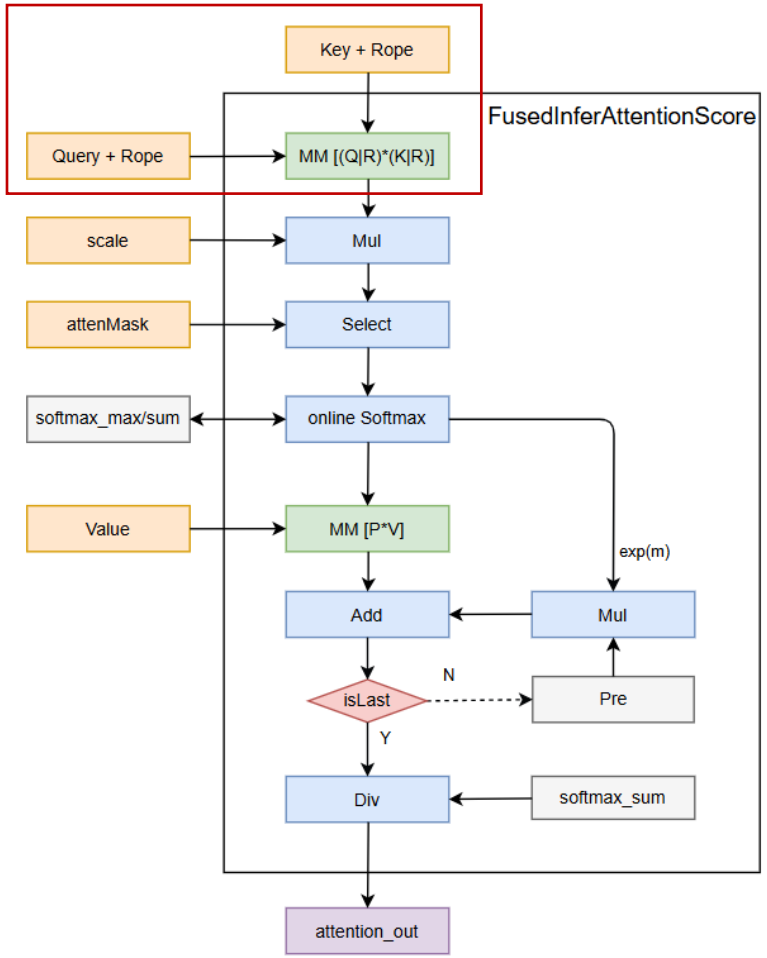
上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

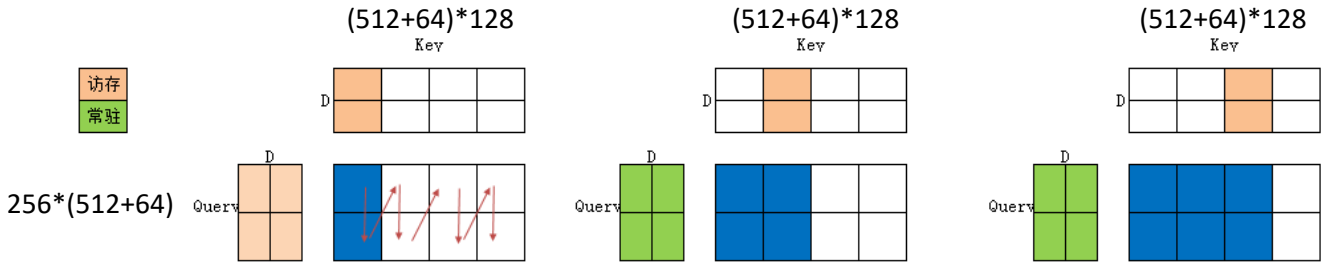
CANN

数据常驻优化

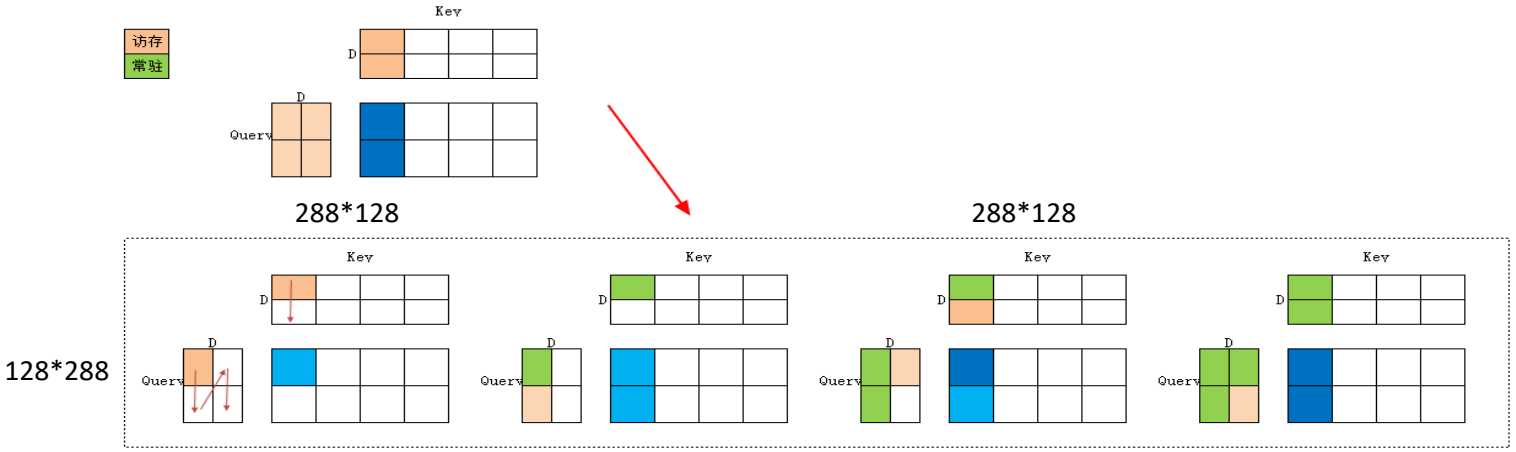


https://gitcode.com/cann

在单个Tiling块的计算过程中，Q+Qr的Tiling块大小为 256*576，K+Kr的Tiling块大小为576*512。计算时将Q+Qr的数据(256*576)常驻内存，并将K+Kr按N轴切成4个基本块(576*128)，分别与常驻在内存中的Query做矩阵乘，减少Query数据的重复搬入

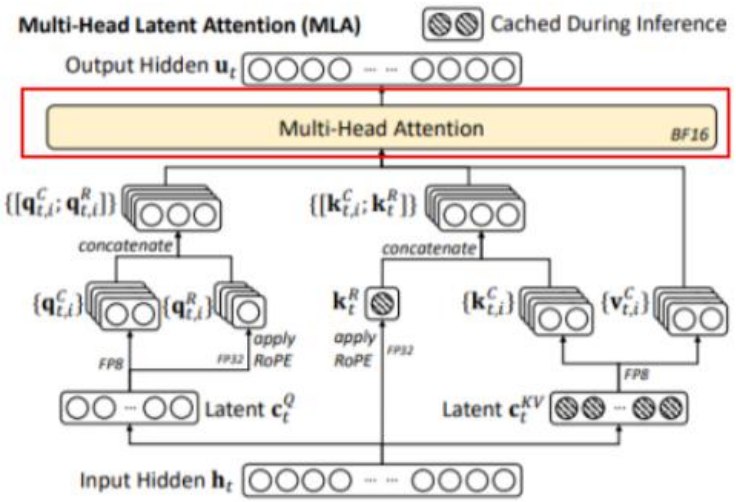


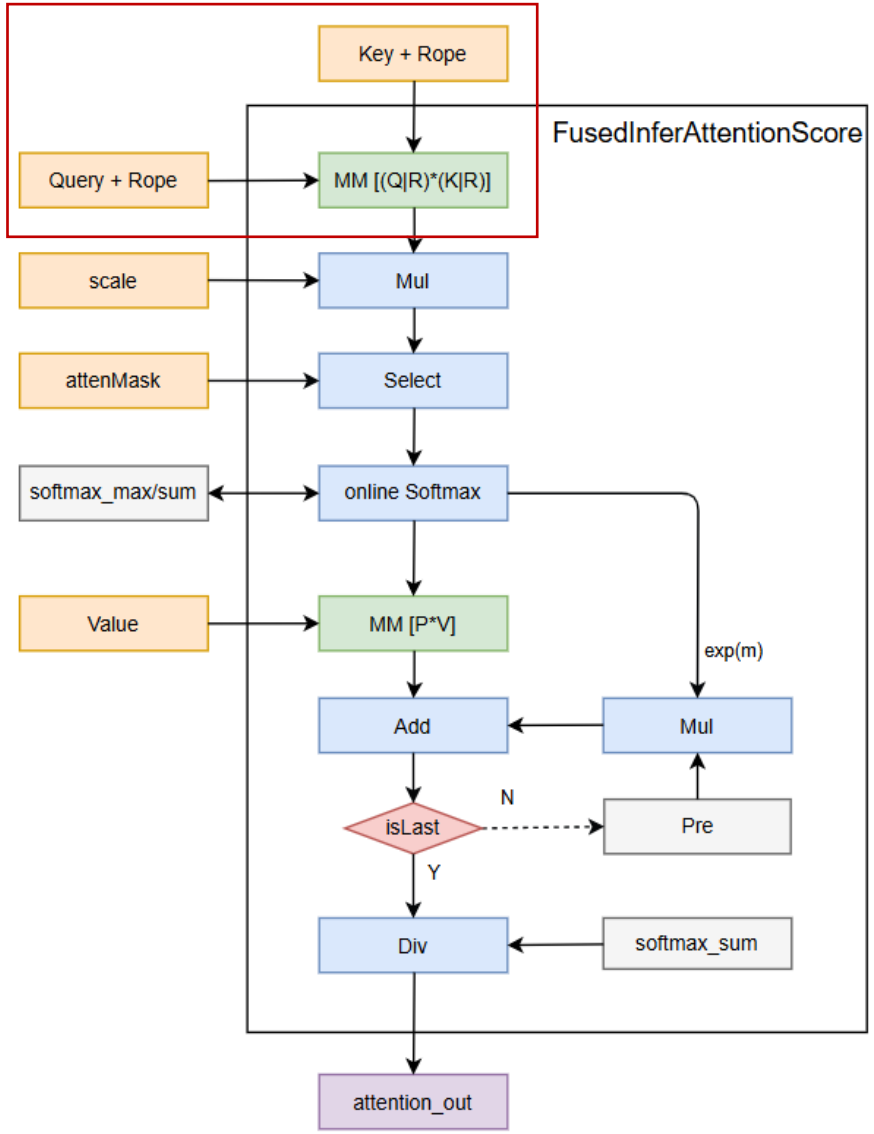
在Q+Qr与K+Kr的基本计算时，将Q+Qr按M轴和K轴均对半切的方式切成 128*288 的大小，将K+Kr按K轴对半切的方式切成 288*128 的大小，并且调整矩阵乘法的遍历顺序，先遍历M轴，再遍历K轴，使得被切块后的Key能做一个小常驻，减少对Key的重复搬入



FIA算子适配MLA

DeepSeek的Decode阶段，使用了MLA技术。使用低秩联合压缩技术降低KV cache存储量；在MLA结构中增加了Rope旋转位置编码。

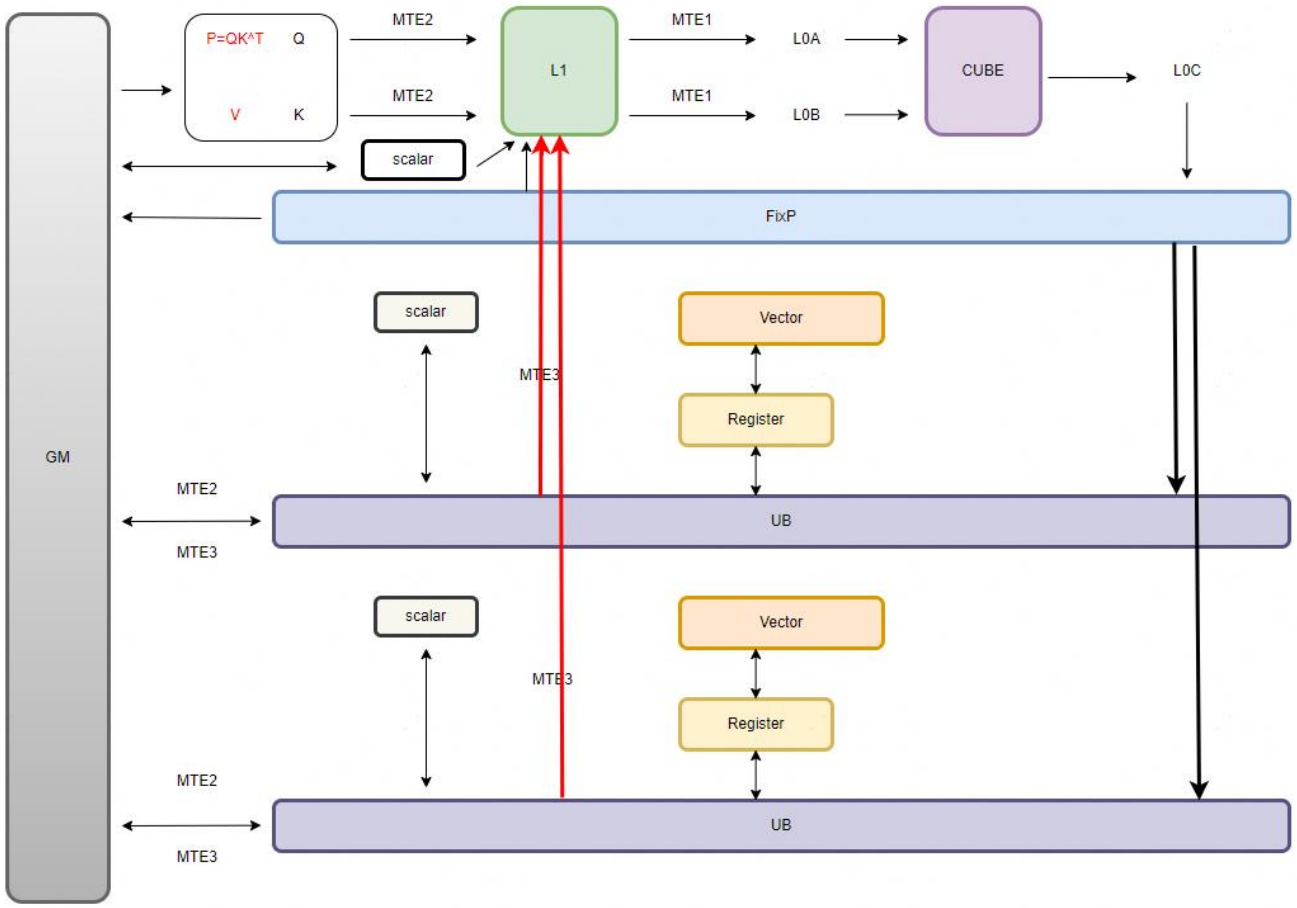




https://gitcode.com/cann

CANN

CV共享通道带来的性能提升



mm1: Q K矩阵相乘 （GM->L1->L0A/L0B->MMAD->FIXPIPE -> UB）
Q为左矩阵，从GM通过MTE2搬运到L1，然后通过MTE1搬运到L0A；
K为右矩阵，从GM通过MTE2搬运到L1，然后通过MTE1搬运到L0B；
执行mmad将L0A/B相乘结果输入到L0C；
最后执行FixP将L0C数据搬运到UB；

vector1：主要计算add，muls，softmax （UB ->VECTOR -> UB -> L1）
等待mm1的结果输出到UB；
UB->register->vector：将信息传送到寄存器里通过VF指令进行softmax等计算，结果传送回UB；
由于mm2需要用到vector1中的结果，所以要通过MTE3将vector1的结果输出到L1(红线所示）；

mm2：P V矩阵相乘 （L1->L0A/L0B->MMAD->FIXPIPE -> UB）
P(P=Q*K)为左矩阵，等待vector1结果输出到L1后，通过MTE1搬运到L0A；
V为右矩阵，从GM通过MTE2搬运到L1，然后通过MTE1搬运到L0B；
执行mmad将L0A/B相乘结果数据读取到L0C；
FixP将L0C数据搬运到UB；

vector2：flashupdate （UB -> VECTOR -> UB -> GM）
等待mm2的结果输入到UB；
UB->register->vector：将信息传送到寄存器里通过VF指令进行flashupdate(muls,add,div)，结果传送回UB

如果是flash最后一轮，结果通过MTE3输出到GM

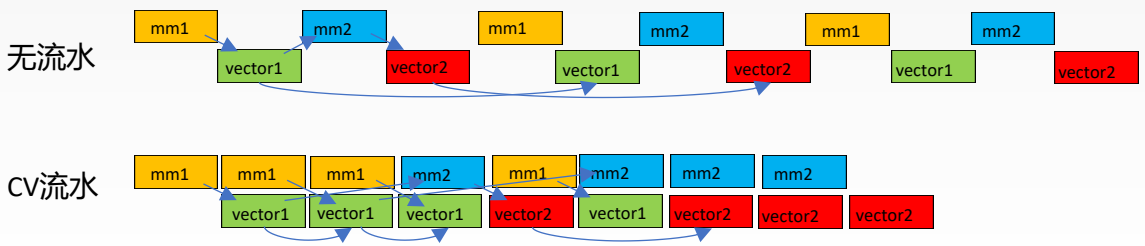
FIA在计算过程中，利用CV之间的通路，将Cube（mm1、mm2）计算结果存储到Vector的UB上，将Vector的计算结果存储到Cube的L1上，避免中间数据进入到GM而增加片上内存的访存量

CANN

FIA算子常用优化手段

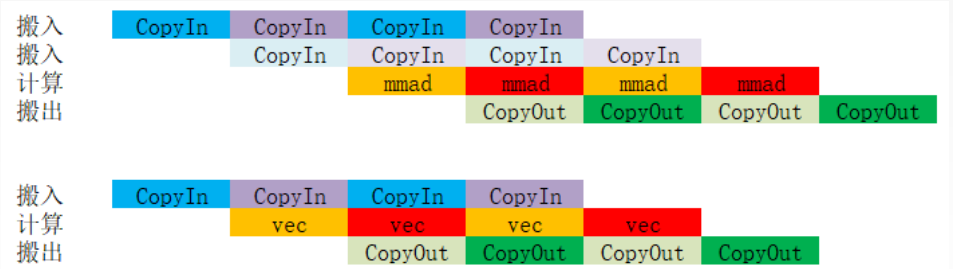
CV紧耦合带来的性能提升

FIA涉及Cube和Vector的计算，通过调整切块后mm1、vector1、mm2、vector2的执行顺序，将没有数据依赖的mm计算提前执行，从而使得没有数据依赖关系的Cube计算和Vector计算能够并行执行



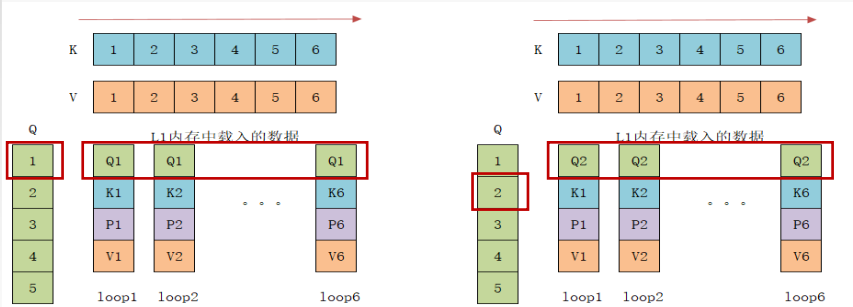
FIA的RegBase的融合方案带来的性能提升

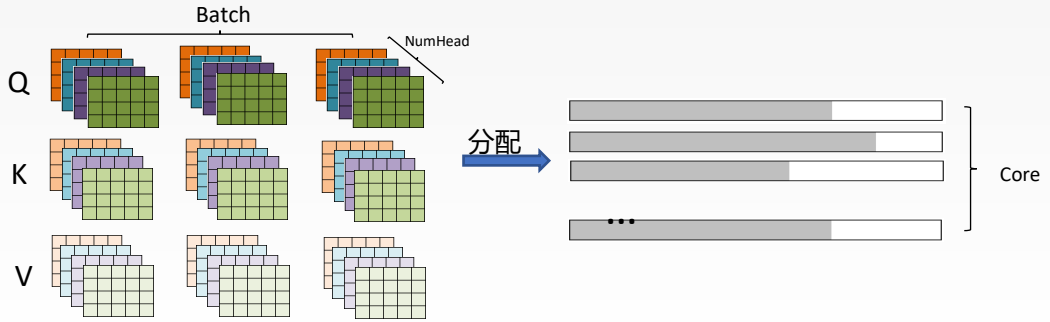
Cube和Vector上的计算过程，都涉及数据搬入、计算、数据搬出流程，使用Double Buffer技术，使得数据搬入、计算、数据搬出能够并行执行



LayOut排布优化

FIA对Q、K、V切块后，在遍历KV的时候，可以将Q子块常驻在内存中，K子块按遍历顺序依次载入内存，与常驻的Q子块分别进行mm1计算，避免Q子块的重复访存



更优的负载均衡优化方案

对Q、K、V按Batch、NumHead、Seq切块，将切块后的Q、K、V，按计算量大小，均摊到所有的核上进行计算